德塔三阶傅里叶计算定义: 一般指将线性的时序语音波进行傅里叶变换, 此时的波为 频率域波, 通过简单的噪声频率过滤后, 让后再进行第二次傅里叶变换. 于是输出的时序波结果会非常的均匀和格式化, 产生优美的平滑间隔峰区间, 于是将此时序波第三次傅里叶变换, 再次得到的频率波输出具有明确的间隔峰区间生物特征标记. 用于德塔语音识别.

定义人 罗瑶光

DETA three times FFT/DFT of higher-order, means a wavelet which makes a transformation from the timer sequence waves to frequency wave, a then makes a noise frequency filter first. At this time, does again a wavelet which makes a transformation where from frequency wave to timer sequence wave. An observation of sequence wave is more harmoniously and uniformly. Also, can do more valuable filters here, therefore, finally does the third wavelet which makes a transformation where from the timer sequence wave to frequency wave again, the output can be a raw data collection of voice recognization and Bio-target.

Author YaoguangLuo 稍后优化语法

具体描述: 智能声诊断视频描述

关于声音噪声处理的描述: 作者通过java sound的jdk开源语音包api 进行声卡录音处理, 通过麦克风物理设备来进行 data readline 函数 record 自然界声音, 这时候声音是一串串的1维 double 和 float (具体看api的版本配置)线性数据, 于是作者将这些线性的 数据进行每 1024 个数据循环进行一小段小段的提取进行

第一次离散傅里叶DFT变换(快速傅里叶FFT也可以, 只是在处理偶数和对称数列比较好), 这时候第一次 时序波变频率域波的 波形变出来了, 于是作者进行更进截取低频的主要区间频率, 依次进行了拉伸, 和高斯平滑处理, 然后把处理过的频率波进行波峰线提取, 然后将波峰线进行极值化, 更进向左平移对齐(方便格式化取值), 进行

第二次离散傅里叶DFT变换, 于是一个稳定的平滑的时序波就出现了. 作者将这个稳定的时序波进行 单个的间隔峰裁剪, 因为只要一个震荡周期区间即可标识特定周期循环出现声音, (不需要多个 时序的震荡周期, 当然多个可以统计增加精准), 于是开始将这个裁剪拉伸的单个间隔峰时序波 进行

第三次离散傅里叶DFT变换, 这时候频率就比较稳定了. 这个3次傅里叶声音计算过程, 可以用来做声音标记等, 广泛应用于声音类的工业场景. 作者的测试实例主要来自作者(罗瑶光)自己的AOEIU 元音发音记录实例.

描述人 罗瑶光

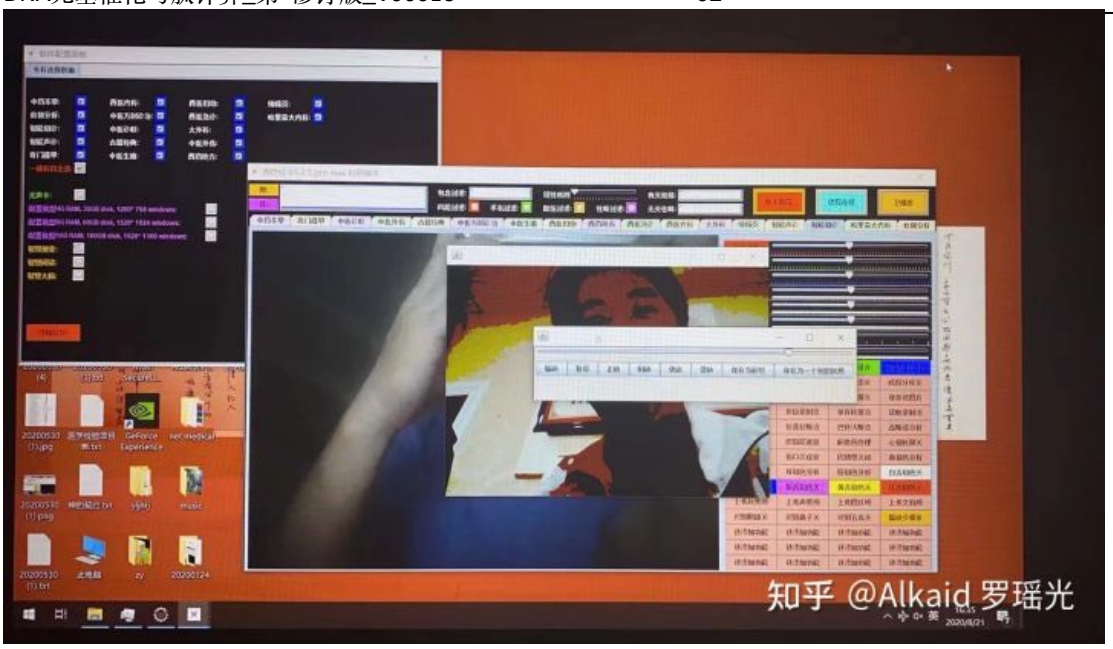
数据的可计算格式场景

1 图片的操作

. refer page 214

2 像素的操作

. refer page 见视觉



智能相诊多媒体图片描述

作者给养疔经的智能相诊设计了视频暂停, 录入的功能, 首先, 视频录入方便了养疔经进行视频快放和倒放的功能, 这样视频的多媒体处理能力得到了加强, 其次方便截图和截图的图片进行像素级别操作, 如单一帧图片的像素画图和像素标记, 标记的图片帧又能融入视频中, 复合加强视频多媒体处理能力. 描述人 罗瑶光

3 文件的存储.

refer page 214.

4 语音的处理.

refer page 见听觉.

格式化卷积计算在仿生听觉中应用

1 滤噪计算 高斯1D

, median refer page 206, 213, 260.

2 频率变换

傅里叶, 快速傅里叶 refer page 258

关于ETL 视觉流的应用描述

首先, 我们导入一张图片, 如作者的大头贴, 图片因为是用低质量的摄像头进行拍摄, 虽然很清晰, 但是色阶比较暗淡, 作者便用计算机视觉研究生教材的像素拉伸思想进行java编码, 将一张0~255的色阶图片拉伸到20~230左右, 于是得到棱角分明的图像, 因为太多分明, 于是用高斯卷积来模糊处理下, 这样图片就是棱角分明而且视觉平滑, 高斯平滑后进行索贝尔mask梯度(mag+dir)卷积计算, 于是可以看到眉毛和眼睛的梯度差如灰色和

白色的辨别. 最后进行emboss 浮雕卷积计算. 作者用这个图片 有力的 举例论证 ETL + 卷积视觉 能够广泛适用于其工业应用场景.

备注作者的计算机视觉绿皮书是没有java代码的, 代码来自作者把Rein hart布置的课后作业都做了一遍. 作者联想自己能有今天的学术造诣, 不仅仅是在 印度基督大学进行完整的系统的学习和研究教材, 甚至连加州路德大学 卡拉森的数据库作业关于UC戴维斯大学的题型, 特别是作者贴在qq空间的所有加州路德大学的作业答案, 答案全部来自作者自己独立自主的辛勤劳动.

描述人 罗瑶光

Implements a visual ETL pipes.

At the first, we might input a picture with authors portrait, to do the visual ETL pipes. Due to the catching with a lower scaled camera, so that the picture was not clearly, means a little bit dark and fussy on it. Then the author started a stretching procedure to make it more distinct, for example stretched the pix, which ranged from '0~255' to '20~230', then he got a distinct picture, and continued to do a Gaussian procedure to make it frequently and smoothly. Finally did a Sobel mask procedure with Mag and Dir. The gradient output of gray and white, where could be a 3 dimensional feature as an input array-value of Emboss procedure. This visual ETL pipes could prove that it widely was used in the real word and around the industries. The author appreciated his well education at CLU, and enjoyed his computer vision class from his teacher Dr. Rein-hart. By the way, He also had finished all of his Database homework of UC Daves by his own abilities from CLU, where he pushed commits on QQ Weibos in an early time. And enjoyed his database class from his teacher Dr. M.Klassen.

The author YaoguangLuo 稍后优化语法.

观测特征识别-视觉计算

1 德塔的视觉主要包含常见2维卷积滤波函数

. refer page.

2 边缘计算

索贝尔凸蚀, 索贝尔梯度, 索贝尔向量, 拉普拉斯refer page 218, 212.

养疗经 肽计算版本 智慧搜索

1 一些医学的归纳关键字索引, 可以进行十六元基变换, 进行不断地归纳和细化, 索引不但可以进行新陈代谢缩进, 同时归纳的索引也不会去阻碍sonar等测试方式去重. 减少源码体积的同时, 还可以枝叶化函数索引分配优化, 进行有效地多函数call计算耦合调度加速.

2 另外在 data swap 层面可以将搜索关键字进行元基细化成肽三元编码, 搜索的条件也会细化到更微级别, 加强搜索的匹配效果.

结合 智慧相诊 应用实例 文字描述

作者将 医学骨科教材的疾病图片进行格式化存储, 用于搜索和像素展示, 展示的过程中集成了卷积像素处理的函数. 如将图片搜索页的数据进行智能相诊页综合处理, 这时候 emboss的浮雕计算可以描绘出三维的立体感, 也可以用绿色像素抑制来观测黄色骨架结构. 这是一个自主游标操作的环节. 可自适应. 描述人 罗瑶光

3 凹度计算

emboss浮雕, 索贝尔mask, refer page 204.

4 频率计算

傅里叶时序域, 傅里叶频率域, 哈尔计算, refer page 258, 211.,

5 腐蚀计算

膨胀计算, 侵蚀计算, 均值计算, 高斯计算1D一字, 高斯2D十字. refer page 202, 204, 205, 206.



图片左上角的独角兽等图片来自2014年 罗瑶光先生通过谷歌浏览器上搜索复制下来的图片

德塔Java卷积算法在 索贝尔家族的 场景计算中描述.

主要描述下 索贝尔凹度, 凸度, 梯度, 向量四个属性. 作者在路德上课的时候比较系统的学习了索贝尔算子原理, 曲力学原理和常见的mag, dir应用边缘求解场景. Reinhart给作者出的考试题目是关于m, d的mask梯度求解实现. 这些都很好解释, 如计算机视觉的绿皮教材中有非常详细的单词描述.

作者在2018年后设计红外监控系统. 计算索贝尔算法在实际的工程应用中, 得到一些经验归纳. 发现索贝尔的实际用途不是书上介绍的那么简单. 确切的说, 索贝尔算法属于力学范畴, 只是恰好在计算机视觉中展示比较直白, 结果明朗, 被人所接受. 确切的说索贝尔算法描述的是一种自然场力. 我来做着研究描述. 当我们设计了mag 和 dir, 两种, 一种为规范的折叠曲线, 我们可以理解为视觉边缘轮廓, 一种为梯度曲线, 我们可以理解为视觉向量雕塑. 在计算机视觉中两种像素集 mask 可以得到边缘的立体视觉, 而这只是对索贝尔的最简单应用.

索贝尔的mag的折叠边缘, 在力学中, 属于挤压的意思, 而 dir的梯度在力学中属于张力和结晶. 不仅挤压有规则, 结晶体同样有花纹. 这两种力的表达可以将图片进行光影的分布进行有效地三维计算, 在mag的边缘部分以挤压力差值明显, dir的梯度可以确定是 挤 还是 压, 确定三维力学的向量. 这些向量于是可以表现为 张力和曲力, 也可以表现为 挤凸蚀 还是 压凹蚀.

上面是索贝尔的力学简单表达的一种规范, 跟进力学计算中, 如果将索贝尔的力进行微分, 那么这种区间内的向量数量, 表达含义将非常丰富. 如场力的稳定状态, 力场的不规则求解 场力的流动性, 物体的内部状态, 很多人会问到这 力怎么能用像素来表达呢, 作者举个例子, 像素是扑捉光影, 光其实一种能量, 确切的说 图片 其实 就是一张能量图. 这就好理解了. 能量是可以进行力学转化的. 如作者的头像进行索贝尔mask, 可以清楚得到, 作者眉毛和眼睛的力学表达是眉毛上凹下凸, 眼睛上凹下凸, 的向量表达, 白色部分代表能量的聚集. 可以理解为索贝尔计算表达的是作者头像图片中的不稳定区间能量扑捉, 只是刚好体现在棱角边缘而已. 而这种能量扑捉能进行3维化.

描述人 罗瑶光 稍后优化

DETA Java convolutional sobel algorithms and Its applications.

The author mainly implemented four sobel attributes of concave, convex, gradient and vectors. Since he had been learned a sobel of 3-dimensional distinction by calculation of Mag and Dir (magnitude and direction). The author developed RF infrared monitor by using Java CV at 2018, to get the sensor catching. According to this experience, the author considered the ability of Sobel algorithms, which belonged to a mechanics-domain. Means Sobel algorithm could explain the randomness of chaos, a mere coincidence at here, computer vision of edge detection, could obviously do a well observation. Absolutely the Mag and Dir of Sobel algorithm only just did a simple procedure of 1 edge-detection and 2 vectorial portrait-sculpture here.

The author considered the magnitude of sobel, means a squeeze pressure or tension in a mechanics domain. And a direction could explain well of Its regulation and pattern. These two factors could calculate the arrangement of ray, distinction of winding-tension, and contrast of concave-convex. To determinate the vector of 3D mechanics.

Promoting on a higher layer of mechanic-computation. In case of mechanical separations from magnitude of sobel, which into a differential level, could explain well of stable status, fluidity and inner phase. A question about asked by some one else: what and how does the connection between mechanics and pixes? The author considered a pixes-picture in fact was a reflection of energy pixes, the energies could transport to a mechanic. A higher ranged scales of Sobel pixes are, a higher entropy and energy-increment could be expressed. A merely show a simple way of pictures was a well edge detection on pictures.

The author YaoguangLuo. 稍后优化语法.

微分催化快速排序优化

1 德塔的排序

作者早期2009年设计《算法导论》 黑皮书, 北邮出版社有其 数据结构 影印教材 的 快速排序4代, 进行了10年优化, refer page https://github. com/yaoguangluo/Data_Processor/blob/master/DP/sortProcessor/Quick_4D_Sort. java

2 优化过程归纳,

逐渐地形成了一个微分催化排序体系. refer page 247, 248, 250, 658, 下册134,

3 左右比对算法优化,

小高峰过滤优化, 缺陷峰归纳, 催化算子优化, 离散逻辑优化. refer page 658, 下册134,

左右比对算法优化, 一般指在不对称的数列中, 为了寻找对称性观测面, 作者设计了一种比较简单的方法, 如将数列逐渐拆分, 取出拆分后的小数列的初值和尾值进行比较, 作为一个参照点, 用于躲避计算高峰. 测试发现具有强大的实用性.

小高峰过滤优化, 一般指为了躲避内存计算高峰而导致的延迟, 卡顿, 死锁, 堆栈溢出等问题 而设计的一类高效率算法集合.

缺陷峰归纳, 一般指计算数列在不断地拆分中的中值基偶问题导致了变量, 算子, 函数的使用频率不对称而出现的一系列蝴蝶效应问题集的归纳.

催化算子优化, 一般指 计算的中间过程中 因 变量, 算子, 函数的使用频率 不对称, 不稳定导致的各种问题, 为了解决这类问题而 进行的不断 的对 变量, 算子, 函数优化与校正过程.

离散逻辑优化, 一般指 对 变量, 算子, 函数优化与校正过程中 通过离散数学, 迪摩根定律, 等客观存在的逻辑定律进行 不断优化与校正过程. 定义人 罗瑶光

Implements of DETA catalytic sort.

The raw material of DETA catalytic sort was Quicksort4D, 2009 from <<Introduction to Algorithms>>, and Its refer page:

http://staff.ustc.edu.cn/~csli/graduate/algorithms/book6/chap08.htm. Theory

https://oi-wiki.org/basic/quick-sort/. Theory

Its optimizations gradually became a catalytic sorting system. Such as left and right marching, inner and small peak filtering, defect peak induction, and discrete logic optimization etc.

Left and right marching, a mainly in an asymmetric and un-regular array, in order to find a symmetric observation. The author separated main-array out of inner-array sections and where found a max value between a Left prefix and right post-tail. Then did conditional performs to avoid the high peak of computing. It seems much more important about reliabilities.

Inner and small peak filtering, a mainly collections of high-effect algorithms, which in order to avoid problems such as the CPU lazy, net gate laggy, DMA crash and Memory out of stack.

Defect peak induction, a mainly problem's induction, in an asymmetric and irregular statement. Means a butterfly effect of Defect computing problems of middle sets, values, functions and asserts.

Catalytic sets optimization, a mainly a collection of optimizations to resolve this defect computing problem of above middle sets, values, functions and asserts.

Discrete logic optimization, a mainly a collection of algorithms to provide a catalytic optimization about above middle sets, values, functions and asserts. These algorithms included Discrete Mathematics, Demorgan Algebra, Objectives and Nature-Definitions.

The author YaoguangLuo 稍后优化语法

4 目前代表作为TopSort5D 极速催化排序.

refer page 下册134

两种比较领先的排序思维对比

罗瑶光手稿

今天我有阅读了一种这几年比较流行的快速排序思想, 基于中轴分离进行左右分配的数列递归排序法. 地址: github 上的 dongxingrong. 我仔细的研究了下, 发现有几个地方可取, 于是进行评价:

地址: https://github.com/dongxingrong/QuickSort/blob/master/src/QuickSort.java

我思考了很久, 在同频算子减少的方向中, 直接减少 temp 的 swap 变量是一种先进的思想. 但是利弊交错, 我进行了和我的 4 代小高峰过滤快排做了详细对比:

1000 万数列排序	瑶光高峰过滤	xingrong 中轴分配
算子减少	有	有
条件减少	有	有
离散效率	高	中
条件过滤	有	无
高频降解	无, 高频优先	有
计算性能	高	高
代码缩进	有	有
平均高峰先排耗时	1137 毫秒	1186 毫秒
平均分配先排耗时	1072 毫秒	1178 毫秒

注解: 这里的函数效率是指单个函数运行的计算消耗, 包括堆栈消耗, 内存阻塞, 计算耗时等因素. 计算性能指的是数组的最大长度, 单位消耗的时间, 内存大小.

比较可以发现, 因为递归的小高峰的存在, 中轴心分配所需要的位移变换成为了必要执行功能, 增大了开销, 这种开销可以最大限度的达到 balanced 递归逻辑的层数, 但是对于指数方的多维算子条件缺少了过滤层, 所以, 性能耗损大. 这种排序思想是完美的, 为追求完美, 丢失了一些东西, 比较可惜. 但是这种思想需要进行解析, 在很多算法应用领域有宝贵的价值.

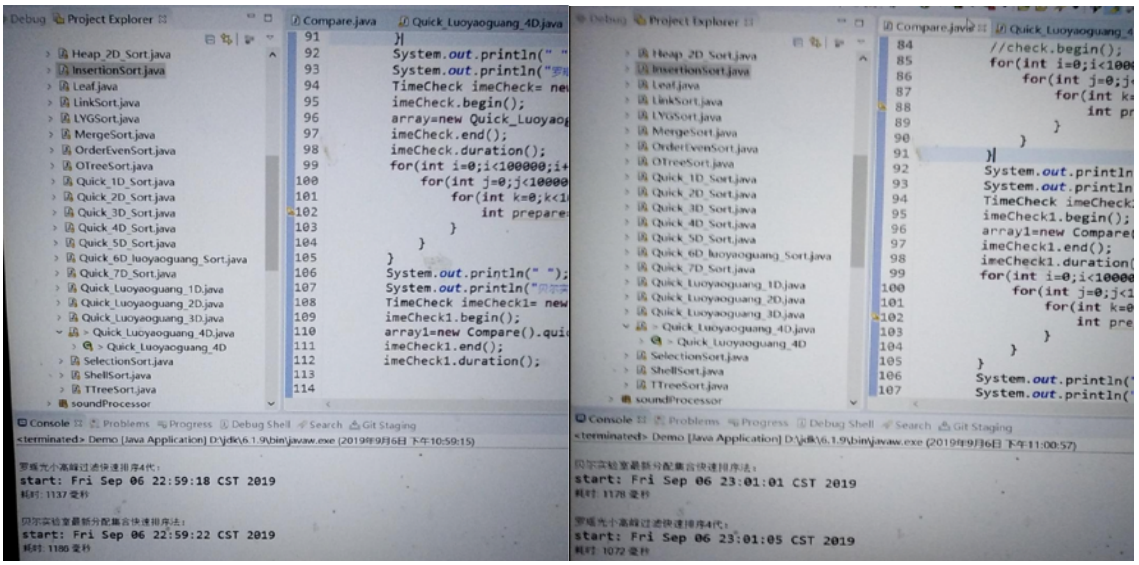
关于看了dongxinrong的无condition条件排序对比, 作者当时随便github搜 天下第一排序法, 世界最快排序法 等关键字 搜的, 目的是进行速度pk, 看自己的算法斤两, 今天看了下(20220330)思考一个问题, 排序中的剔除IF 条件, 作者认为, if 用中文理解, 可以定义为某种事件, 确切的说是某种确切的事件, 因为是确切, 所以没有概率, 没有条件, 必然执行, 或者执行了也不会影响本身, 可以理解为容错率, 这种特性, 在一些场景中, 一定有可以用得到的地方. 作者期待中, 希望能够遇到. 后面必然是一个世界.

描述人罗瑶光

下面的 4 张图是堆 1000 万个随机数进行排序的比较耗时测试.

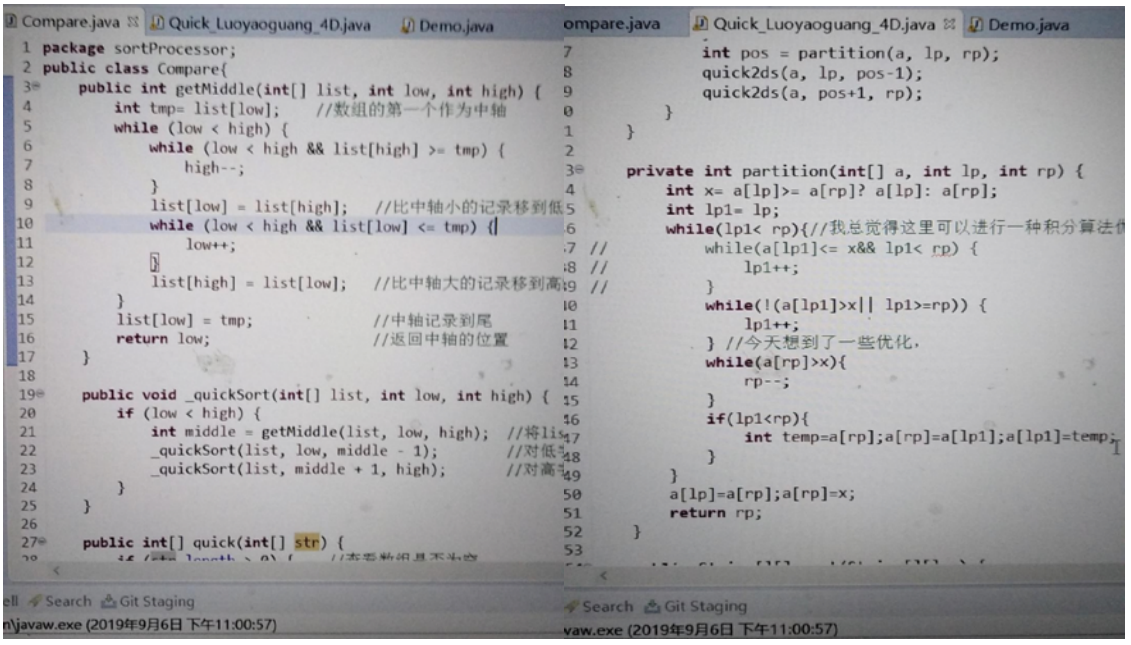
图篇的pk测试对比中出现的贝尔实验室的快速排序是标准的 算法导论中出现 4代排序算法.

对比的作者的算法, 当时已经添加了 小高峰左右比对法, 因为图片是很早版本, 还没有进行离散优化 和 算子过滤优化. 但当时性能对比优势已经明显.



上图为作者2014年和贝尔atnt的霍尔排序比较

下图为作者和Github推荐的快速排序比较



数据结构发散-搜索 稍后

1 德塔的搜索计算

主要做一个编码参照, 没有工程用途. refer page 226

2 编码参照

有利于作者在设计图论计算和非线性搜索时候发散思维用途. refer page

3 编码溯源

开始于作者2009年 完成 Rohini教授布置的作业. refer page 我 qq 313699483 有完整作业备份日记.

4 编码用途

对作者研究Hash空间 有发散思维的用途, 如作者数据预测包设计的辅助. refer page

应用

1 TopSort5D 包含深度算子, 包含广度算子, 包含滤波算子.

refer page 下册134

关于极快速微分催化排序的deep 变量溯源

作者早年仅仅只是完成印度基督大学 rohini教授布置的《数据结构》课后作业, 作业做的比较仔细, 当时图论的广度和深度计算体现在神经网络的路径探索中. 举例如balanced tree的搜索与遍历. 和 哈夫曼编码模拟实例. 更多的具体应用如 计算器的复杂四字运算括号识别. 这个基础意识也是作者的top sort5D 极速排序体系的 deep 变量的定义最早来源, 这里有必要描述下. 因为极为优秀的快速排序思想 毕竟也有个缺陷便是疯狂的吞噬内存堆栈, 一开始国际比较通用range来消除堆栈吞噬问题, 如《算法导论》的4代已经包含的range函数原型, 作者进行了关联发散思考, 作者严谨地认为如果在工业场景中, range用于确定堆栈的损耗减少界限, 那么这只是对特定一种balanced堆栈分配的界限条件状态有价值, 如果在一些非对称峰中进行切割和计算, 那么range的表达能力便不强了, deep能更好地确定迭代层次, 有效地对堆栈吞噬问题直接进行一刀切, 于是deep + range的组合界限变问世了. 描述人 罗瑶光

Implements of speed differential catalytic sort about Its trace of 'deep'.

In a colleague time, the author did a well learned of <<Data Structure>>, especially in Its domain of deeps and weights graphs of tree-rotation and manipulations, India. The author considered a technology of balanced tree which could be used for a reference to the domain of DETA catalytic sort. For example, deep value. Before the optimization of quick-sort4D, <<Introduction to Algorithms>>, which had already contained a ranged value, to avoid the 'out of memory stacks'. The author considered that ranged value had had only just suitable for a special environment. Such as a balanced recursion. Means that the ranged value wouldn't enough for the author's defect, unbalanced and messy status of computings. Therefore, a deep value could scale the recursion and its hierarchical stacks, which looked like a 'one-size-fits-all approach' of 'uniform limit'. Seem the deep+ range could do well limitations to scale the sorting problem of 'Stacks Out of Memory'.

The author Yaoguang.Luo 稍后优化语法

TopSort5D 版权源码

```
private void processDouble(double[] array, int leftPoint, int rightPoint, int deep) {
    int c= rightPoint- leftPoint;
    if(!(c< this.range|| deep> this.deeps)) {//balance催化减少条件递规深度思想. //流水阀门优化思想.
        int pos= partition(array, leftPoint, rightPoint);
            if(leftPoint< pos- 1){
                processDouble(array, leftPoint, pos- 1, deep+ 1);//减少条件递规深度思想.
            }
            if(pos+ 1< rightPoint){
                processDouble(array, pos+ 1, rightPoint, deep+ 1);//减少条件递规深度思想.
            }
        return;
    }
        int i= leftPoint;
    for(int j= i+ 1; j<= leftPoint+ c; j= i++){
        while(j> leftPoint){
            if(array[j]< array[--j]){//催化波动算子duplication 思想
                double temp= array[j+ 1];
                array[j+ 1]= array[j];
                array[j]= temp;
            }
        }
    }
}
private int partition(double[] array, int leftPoint, int rightPoint) {
    double x= array[leftPoint]<= array[rightPoint]? array[leftPoint]: array[rightPoint];//小高峰过滤饱和催化减少条件;
    int leftPointReflection= leftPoint;
    while(leftPointReflection++ < rightPoint){//催化波动算子duplication 思想
        while(!(array[leftPointReflection++]> x|| leftPointReflection> rightPoint)) {}//催化波动算子duplication 思想
        while(array[rightPoint--]> x){}//催化波动算子duplication 思想
        if(--leftPointReflection< ++rightPoint){//催化波动算子duplication 思想
            double temp= array[rightPoint];
            array[rightPoint]= array[leftPointReflection];
            array[leftPointReflection]= temp;
        }
    }
    array[leftPoint]= array[rightPoint];
    array[rightPoint]= x;//小高峰过滤饱和催化减少条件递规深度思想.
    return rightPoint;
}
```

知乎 @Alkaid 罗瑶光

本人调通的算法导论的quicksort4D算法链接如下, 可直接区别, 再次Refer 快速排序之父 霍尔先生:

https://github. com/yaoguangluo/Data_Processor/blob/master/DP/sortProcessor/Quick_4D_Sort. java

关于 算法中的function call在汇编语言中的消费 效率探索

微分催化排序 罗瑶光小高峰过滤计算版本 版权函数 进行去递归 测试

1原逻辑因为区别递归function call在真实的劳动生产中对计算效率的消费总量，于是尝试用loop + class 和数组来替换function call进行对比观测。

```
private int partition(float[] array, int leftPoint, int rightPoint) {
                float x = array[leftPoint] <= array[rightPoint] ? array[leftPoint] : array[rightPoint];
                int leftPointReflection = leftPoint;
```

```
            while (leftPointReflection < rightPoint) {
                while (!(array[leftPointReflection] > x || leftPointReflection++ >= rightPoint)) {
                }
                while (array[rightPoint--] > x) {
                }
                if (leftPointReflection < ++rightPoint) {
                    float temp = array[rightPoint];
                    array[rightPoint] = array[leftPointReflection];
                    array[leftPointReflection] = temp;
```

关于temp的交换方式的可优化形式，这里可以区分下整数与分数，因为分数的IEEE的最新精度存指压缩不利于bit直接交换，于是介绍下整数的双变量交换模型，两个整数 的2进制比如 101011和010111，可以区别 1010 和0101 不同区和 11同区，采用bit的位计算标记同区，不同区则为非门变换即可。如数字逻辑的XOR关系--A非B+AB非--

```
                }
            }
            array[leftPoint] = array[rightPoint];
            array[rightPoint] = x;
            return rightPoint;
        }
```

2class逻辑 我得到一些观测反馈，new对象在jvm中占用大量的heap，而且速度性能大幅降低。效率跟不上function call。

```
private void processDouble(float[] array, int leftPoint, int rightPoint, int deep) {
        LogSortNode logSortNode = new LogSortNode();
        logSortNode.L = leftPoint;
        logSortNode.R = rightPoint;
        cur = logSortNode;
        end = cur;
        float x;
        int l;
        int r;
        float temp;
        int pos;
```

```
while(cur != null) {
        leftPoint = cur.L;
        rightPoint = cur.R;
        int c = rightPoint - leftPoint + 1;
          if (!(c < this.range || deep > this.deeps)) {
            x = array[leftPoint] <= array[rightPoint] ? array[leftPoint] : array[rightPoint];
                l = leftPoint;
                r = rightPoint;
                while (l < r) {
                  while (!(array[l] > x || l++ >= r)) {
                  }
                  while (array[r--] > x) {
                  }
                  if (l < ++r) {
                    temp = array[r];
                    array[r] = array[l];
                    array[l] = temp;
                  }
                }
                array[leftPoint] = array[r];
                array[r] = x;
                pos = r;
                if (leftPoint < pos - 1) {
                  LogSortNode node = new LogSortNode();
                  node.L = leftPoint;
                  node.R = pos - 1;
                  end.next = node;
                  end= end.next;
                }
                if (pos + 1 < rightPoint) {
```

```
                    LogSortNode node = new LogSortNode();
                    node.L = pos + 1;
                    node.R = rightPoint;
                    end.next = node;
                    end= end.next;
                }
            }else {
                int i = leftPoint;
                for (int j = i + 1; j < leftPoint + c; j = i++) {
                    while (j > leftPoint) {
                        if (array[j] < array[--j]) {
                            temp = array[j + 1];
                            array[j + 1] = array[j];
                            array[j] = temp;
                        }
                    }
                }
                }
                cur = cur.next;
            }
            //System.out.println("tot-->" + flows);
        }
```

3 array逻辑，我也得到了一些反馈，array的大小因排序的总量而成正比，数组上千万同样跟不上递归的function call。

```
private void processDouble(float[] array, int leftPoint, int rightPoint, int deep) {
            if(flows == 0) {
                hashArrayL[flows] = leftPoint;
                hashArrayR[flows] = rightPoint;
            }
            do {
                leftPoint = hashArrayL[flows];
```

```
                    rightPoint = hashArrayR[flows];
                    int c = rightPoint - leftPoint + 1;
                //if (!(c < this.range || deep > this.deeps)) {
                    int pos = partition(array, leftPoint, rightPoint);
                   if (leftPoint < pos - 1) {
                    hashArrayL[fix] = leftPoint;
                    hashArrayR[fix++] = pos - 1;
                   }
                   if (pos + 1 < rightPoint) {
                    hashArrayL[fix] = pos + 1;
                    hashArrayR[fix++] = rightPoint;
                   }
            }while(flows++ < fix);
            System.out.println("tot-->" + flows);
        }
```

测试问题

原函数5000万在苹果m1芯片 2.4g-4核-8线程电脑上没有插电源的状态下测试 需要8.8秒，在去递归后用array 是9.2秒，在去递归后用node class是 10.2秒。我很疑惑 省掉了大量的funtion call 和递归 竟然还慢了个5%效率，于是贴出来以飨观众，并归纳下心得：

//递归在区间计算中能 提高计算数组的堆栈中指针数列寻址效率，但要注意是递归对超过10行源码的function call量级才有一定的效率，

```
            private void processDouble(float[] array, int leftPoint, int rightPoint, int deep) {
                int c = rightPoint - leftPoint + 1;
                if (!(c < this.range || deep > this.deeps)) {
                    float x = array[leftPoint] <= array[rightPoint] ? array[leftPoint] : array[rightPoint];
                        int leftPointReflection = leftPoint;
                        int r = rightPoint;
                        while (leftPointReflection < r) {
                            while (!(array[leftPointReflection] > x || leftPointReflection++ >= r)) {
                            }
                            while (array[r--] > x) {
```

```
                    }
                    if (leftPointReflection < ++r) {
                     float temp = array[r];
                        array[r] = array[leftPointReflection];
                        array[leftPointReflection] = temp;
                    }
                }
                array[leftPoint] = array[r];
                array[r] = x;
            if (leftPoint < r - 1) {
                processDouble(array, leftPoint, r - 1, deep + 1);
            }
            if (r + 1 < rightPoint) {
                processDouble(array, r + 1, rightPoint, deep + 1);
            }
            return;
        }
        int i = leftPoint;
        for (int j = i + 1; j < leftPoint + c; j = i++) {
            while (j > leftPoint) {
                    if (array[j] < array[--j]) {
                            float temp = array[j + 1];
                            array[j + 1] = array[j];
                            array[j] = temp;
                    }
            }
        }
    }
```

2 索贝尔 dir 向量差 区别三维的立体面特征趋势.

refer page 219 因为属于当时作者在加州路德大学的课后作业，本书略。

3 噪音识别.

refer page 720 华瑞集在识别方式主要采用组合频率 特征标识 来识别不同的调制解调震动源属性。



4 小波分离.

refer page 不在此章 涉及鸡尾酒调度, 被略去先

鸡尾酒调度算法涉及他人的版权思想, 本书略.

5 极速商旅TSP.

refer page 538, 541, 547 采用区间微分路径计算, 减少 balanced 指数计算面。

6 股票数据抓取

refer page 不在此章, 261, 263, 264, 266可以处理 股票数据线波.

关于股市数据的应用描述

该软件是作者为父亲做个股票分析系统. 作者首先设计一个矩阵分行显示股票数据, 数据来自作者对雪球, 新浪, 财富, 很多网页的抓取, 抓取的网页不是统一的编码格式, 于是进行了gbk, 2312, utf8等格式编码统一(data swap), 这些double数值于是进行多样化的展示, 这里便用到了数据分析算法的 平均值, 统计, 贝叶斯, 等知识点的简单应用. 左下图图片是作者在新浪网裁下来的, 不是作者自己设计算法生成的. 根据进仓资金进行了傅里叶变换来观测庄家进仓频率, 右下图所示. 另外包括十大庄家, 大资金手笔入仓出仓动态. 有力的证明了作者的大数据分析引擎的金融商业领

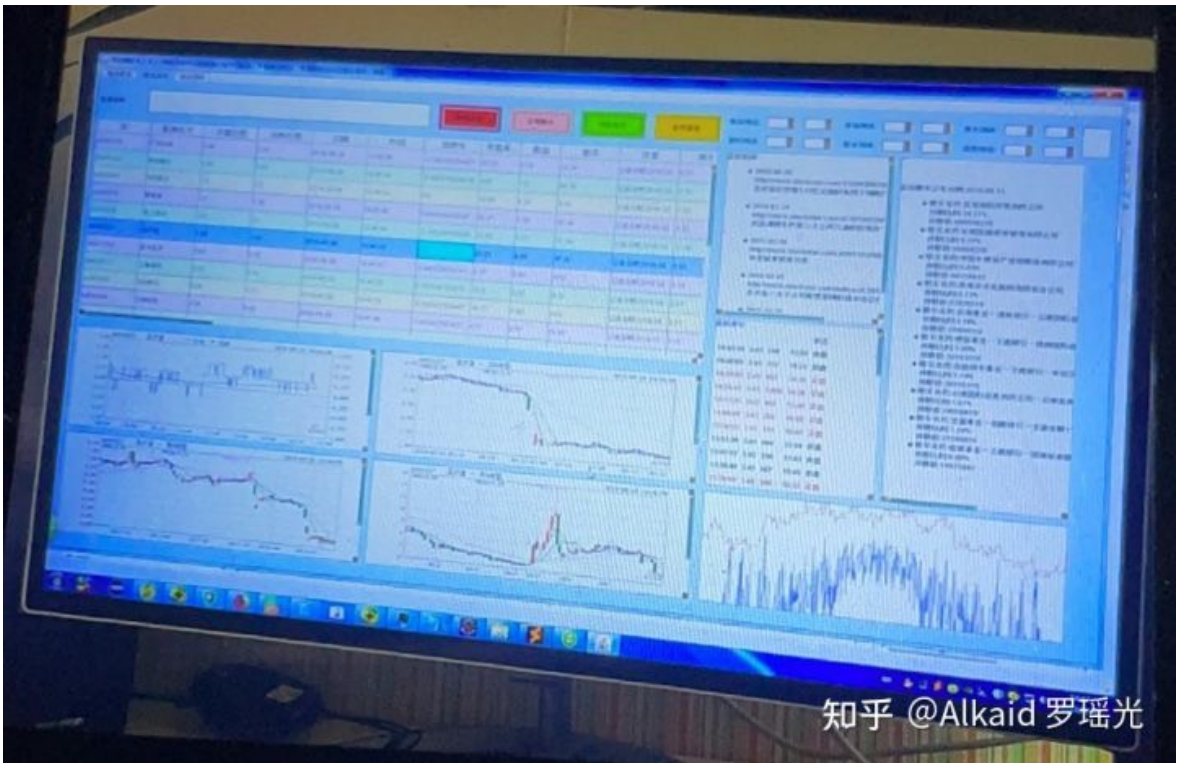
域价值. 作者贡献: 当时发现股市金融入仓的15天比 为 - + -时候 有个特征缺陷, 发现了一个资金当天涨, 当天卖的流入漏洞, 容易被恶意短期操盘. 当时作者群发了此现象. 描述人 罗瑶光

Implements of stock manipulations. The author did a matrix observation by each tupe line of stock data, means row by row observations. These data onto the '雪球', '新浪' and '财富' etc, and these feedbacks on post encoding and decoding were different with each other. For example, GBK, GB2312 and UTF8 etc. The author needed a DataSwap of unifying. Because of the double values of a deversity-demostributions, so the author did the mean-ratio finds, statistics, bayesian and other simple procedures to fulfill the requirements of fund frequently inflows about trading volumes. Then the author used FFT to make a frequency domain of transformations. According to to trace a ten banker and informations about big deal of traders.

Contributions to the world.

The author found a defect of feature, he made a fifteen- and thirty-days calculations about inflows and outflows, which as ratio units by each stock, then distinct the pre-day ratio, this ratio, and post-day ratio, to find theirs 3 ratios assessment of funds increment and decrement. The author found these ratios did a defect show of the day before a day was commonly, but the day was surges, and the day after tomorrow was slipping. And Its 3 ratios assessment mark, was likes '-+-' (decrement, increment, decrement). Then the author used Wechat of sweet00048 bind of '15116110525' to publicly issue this problem of 2018~2019.

The author YaoguangLuo, 稍后优化语法..



极速象契拼音笔画排序

极速象契拼音笔画排序描述:

首先作者设计该算法前已经有了极速排序为基础. 于是尝试将字符串的按拼音和笔画进行象契综合计算来排序. 中文的拼音和笔画, 可以在康熙字典, 词源和新华字典上进行录入. 作者录入的方式比较简单统一为map, 其中文字key对应拼音和笔画value, 进行相应的索引. 这个过程为索引的map预处理. (作者有百度和豆丁文库的词汇购买记录, 如新华字典的中文汉字, 康熙字典, 国家语言等级词汇等等. 目前百度帐号OK、, 但豆丁不是15116110525和qq绑定, 而是用作者的微信sweet00048帐号来绑定的, 2022年3月2号前莫名被腾讯永久冻结, 搞得我当时立即报警， 因为此事非常蹊跷, 所以作者至今也不会解绑, 当然以后也不会解绑, 所以国家依旧能查到.)

作者将这些购买的原材料doc和txt文档进行加工处理生成了2个txt格式的拼音词库文件和笔画词库文件. 于是开始计算字符串比对排序. 作者按通用的拼音进行指针对应的每两行两行进行极快速微分催化, 小高峰过滤, 左右比对法综合按快速排序的方式(目前是top sort 5D)进行从a到z进行char对应int大小来计算排列. 当出现拼音完全相同的字的时候, 于是比对这个同音字的笔画的多少进行深化条件补充排序. 如果在这个时候依旧出现既相同拼音, 又相同笔画的字的场景, 作者便根据字符的char的ascii编码, (utf8, gbk, gb2312 都有相应的文字int编码)来区分. 于是把这整个过程中一些恒定的中间变量全局化作为带精度观测入参, 进行自适应. 目前这个算法的函数一直在进行DNA元基催化编码索引的新陈代谢+二次新陈代谢进化排序优化中. 描述人 罗瑶光

Definition: Speed Sorting the Hieroglyphics and Cuneiform characters by using a sequence of phonetics and strokes.

Firstly, the author had already touched on a good foundation of quicksort and data structure by using C in Christ University since 2008. Then tried to sort the mixed string of Hieroglyphics and Cuneiform characters in a way of phonetics and spelling strokes. And those Hieroglyphics, Cuneiform characters, phonetics and spelling strokes, were arranged and built as object key-maps. The author did a comparative way of those string characters with an ASCII contrast, which means the numeric ASCII of these alphabetic chars. The distinguished workflow as below. While the sort engine found two characters were the same in phonetics, then began to check Its conditional strokes. While the sort engine found two characters were the same in phonetics and strokes. Then began to check the numeric ASCII of these two alphabetic chars.



于是开始思考一些问题

1 关于细粒度的切分问题，排序的单机操作和大数操作有什么异同。

2 排序应用于复杂的工程逻辑如何让计算单元算能更加规则的细粒度

3 细粒度指的是averaged + balanced的函数流操作单位比如call，jump，迭代，递归，变量引用，map等。

4 上述3个问题定位在大学3年级 没有异议吧。。

5 关于卷积的拓扑核的种类，和运算化简规则和可优化模型，定位在研2以上，我这本书不展示先。

第三章 德塔 ETL 人工智能可视化数据流分析引擎系统.

基础应用: 元基催化与肽计算 编译机的神经元模拟机

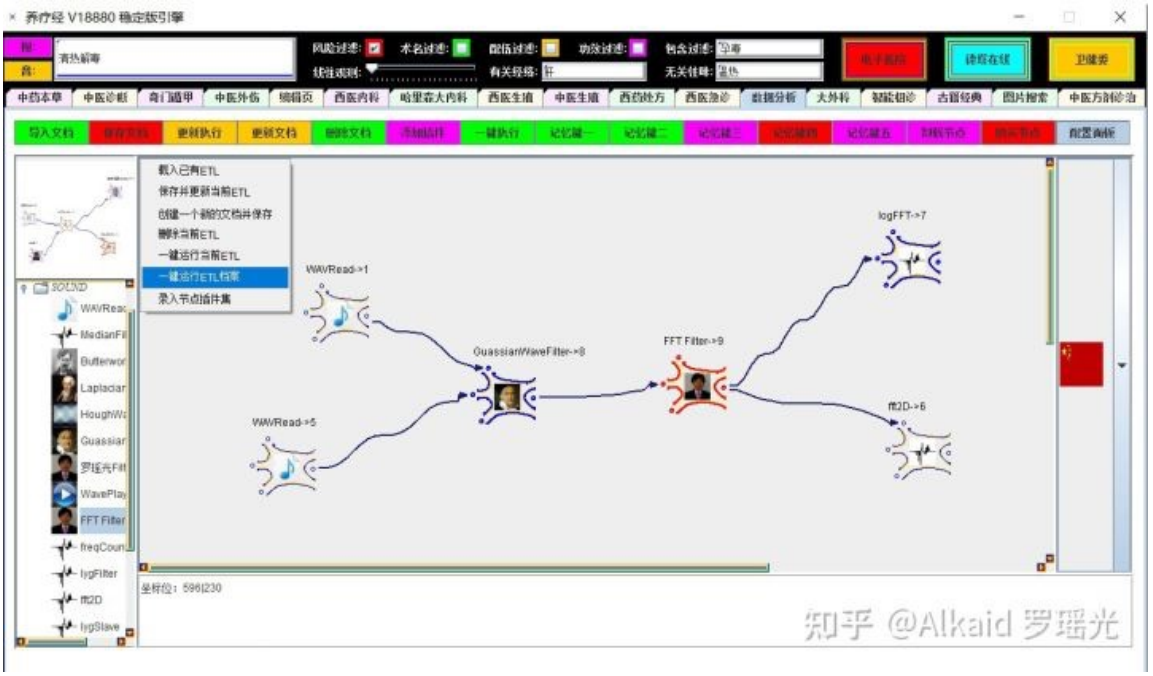
知识来源

, 作者第一次接触节点编码是在上海的章鑫杰公司2009年, 作者的工作任务是基于Knime, 进行eclipse 插件开发Knime的节点, 当时是作者第一次用Java进行工作编码, 作为ESIEE的pascal助理partner, 参与flech元音法语邮件的文本分析.

作者这里要感谢罗瑶林2009年教作者使用eclipse进行1断点调试和knime插件的2单例迭代编码方法, 用于区别java和c, 当时用qq的远程操作. 同时也要感谢章鑫杰在堂弟教了断点调试后第2天也重复教了作者import 和 export 开发包和eclipse进行断点调试, 也是用qq的远程操作.

章鑫杰给了作者和张继斌(后离开上海帆腾去了美国花旗银行就职数据分析员) 关于C#的数据分析软件, 作者记得内含另一种ETL风格的窗口组件.作者在学校接触Weka API, 这是作者见过的第三种Pipeline 节点分析模块, 后设计甲骨文数据库, 又自学了甲骨文的节点流ETL, 之后又在谷歌搜索上自学了Orange和 Kettle ETL, 分享一个秘密, 作者的波浪曲线模拟神经元链接箭头便是来自Orange的灵感,只是Orange更细腻.

这些ETL的文档阅读,安装和使用,丰富了作者在 ETL领域的应用和研发知识. 再分享大家一个秘密,作者在2013年设计ETL Unicorn一开始准备想用C#来设计,模拟章总送的那个ETL（章总的作品+knime的sdk等都是有著作权的, 可以比对源码逻辑, 因为我的源码全部也申请了著作权并全部开源了, 避嫌）的样子,而作者的计算机视觉作业是Java写的API,最后,作者用Java写ETL Unicorn了. 作者一边设计,一边教旁边的李妙环,内心很有成就,,引擎界面一出来,作者2014年去波音面试给Andrew. K. Yoo先生展示了下,后上班(因涉及军事,本人仅仅面试见了Andrew 1次面,从未在波音任职,也没有和波音员工私下接触过1次面,之前的领英号我有加他. 盲分离声音邮件材料谷歌上一搜一大把,不是Andrew给作者的.）去了就淡忘了. 因为养疗经的设计,2019年又重新开始ETL优化编码. 另外,感谢下作者人生第一次用batch line 来执行Java调用exec批处理脚本指令,是章鑫杰教的. 罗瑶光



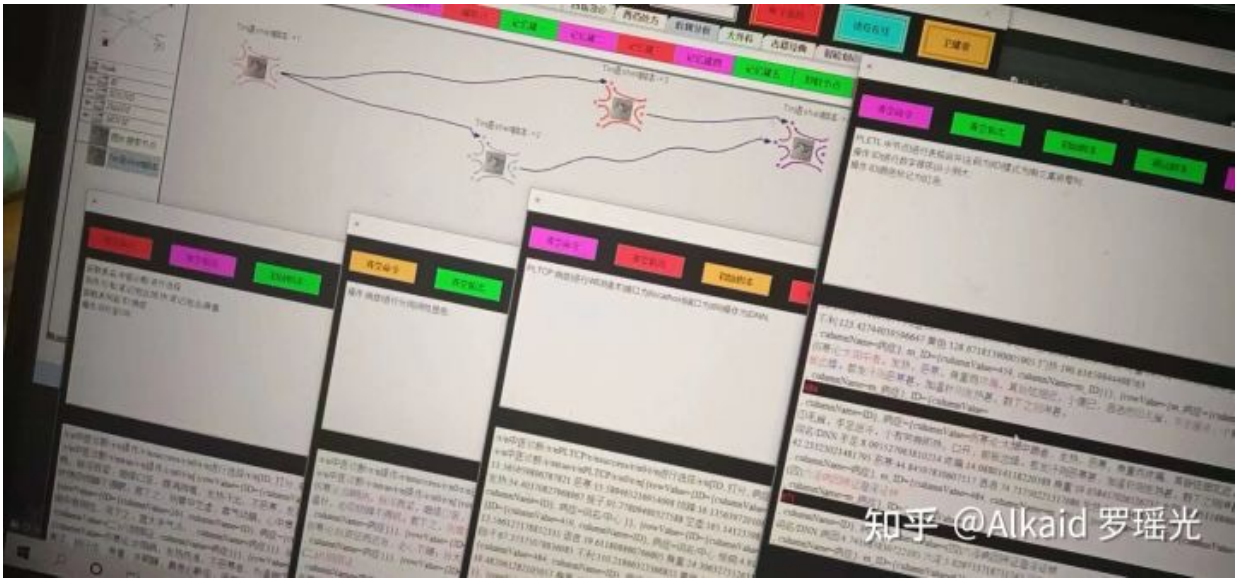
1 德塔ETL 又叫 ETL UNICORN, 是一个数据节点流计算的可视化操作工具. refer page 267

2 最早由作者在路德大学设计java卷积视觉包, 为了方便 像素矩阵流的流水观测. refer page 186

3 德塔ETL采用APPLET, 可以嵌入在网页上作为 rich web架构, 与flash应用相似. refer page 287

4 德塔ETL已成为当前的PLETL, 和元基花模拟神经元计算的基础组件. refer page 774





ETL 节点流界面

1 德塔ETL 的界面

采用Jsplitpane分区 主要包含节点显示树区, 节点画布操作区, 计算状态反馈区 和系统配置区. refer page 286~

2 节点的显示区

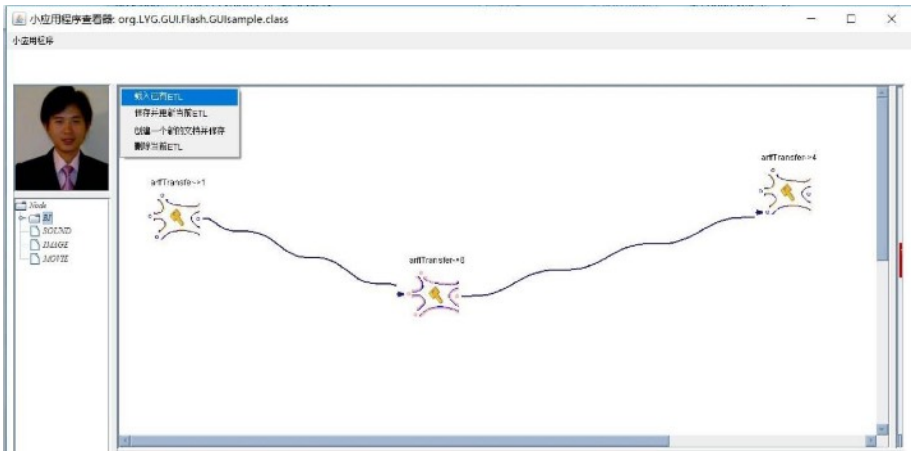
采用Jtree进行鼠标操作, 左键选择, 右键弹框. refer page 286~

3 画布操作区

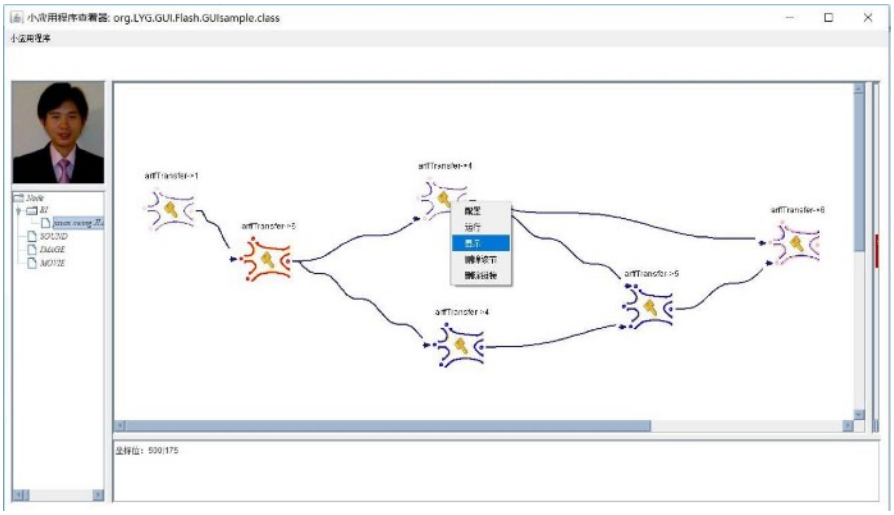
的 节点采用画线描点实现, 操作为左键拖拽, 右键连线和弹框. refer page 311~早期的节点处理界面弹框 设计成inner 弹框模式作者发现关闭按钮被屏蔽了, 于是就改成frame组件跳出canvas画布来显示节点处理界面.

4 状态反馈与系统配置区

用于实时了解ETL的运行状态. refer page 见jtextPanel,最新 养疗经 版本, 作者将状态数据的jtextpanel 写在了元基枝全局变量进行集成, 如果要单独将etl做插件使用, 要区别这个 函数, 可以改写下.

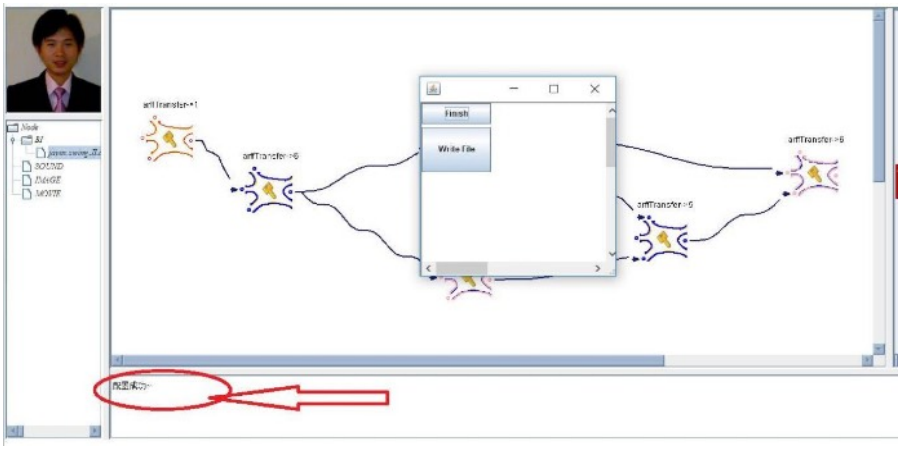


ETL引擎的设计逻辑架构图



ETL引擎图的的鼠标操作逻辑描述

作者采用一种极为简练的傻瓜操作方法对引擎画布有效地鼠标操作, 首先, 在节点的树形



界面操作, 鼠标左键为确定当前选择, 右键为当前选择的功能展开. 举例 鼠标左键点击选中一个节点, 右键点击于是出现弹窗, 点击add添加, 于是右边主画布画出节点图形.

其次, 画布的节点操作, 鼠标左键的功能为选中一个节点, 持续鼠标左键按下可以进行节点的移动拖拽功能. 鼠标右键的功能包含非移动拖拽和移动拖拽, 在选中一个节点后, 非移动拖拽, 可以弹出节点操作功能jframe面板, 如配置 configuration, 执行execute, 展示 view, 删除当前节点 delete, 删除当前节点的关联链接箭头线 delete. 鼠标左键点击功能项可以进行节点详细操作. 关于右键的持续按下移动拖拽为节点的箭头链接拖拽操作, 指在非移动节点的情况下, 进行拖拽所指引的目标节点进行连线. 细节为右键拖拽需要将鼠标指向 节点的关联点区域, 如图中节点左侧的上中下链接点.

描述人 罗瑶光

Implements of mouse manipulations of ETL unicorn.

The author adopted a succinct way of effect mouse manipulations of ETL unicorn. To keep It way of simple, firstly built the tree manipulations where to make a classify of ETL nodes. Once did an expanded window to add the node into a canvas board by mouse clicks. The mainly canvas will present this related node branch and Its manipulations by mouse-click. The left-mouse-click could well choose the ETL node, to make a moving and dragging of the related node. The right-mouse-click could well drag the ETL node arrow, to make a line-link, which connected to the object node. The details of line connection, means It kept an endpoint of arrow which aimed to the hook-area of each node. For example, the node in picture, which contained three hook-areas of up, middle and bottom on left side. Also, the right-mouse-click with non-moving condition, could well click out a jumped JFrame component. This component contained four operations of configuration, execute, view and delete, to the continued manipulations.

ETL unicorn 计算节点的设计描述

作者一开始设计etl节点已经有了基于knime的sdk应用近3年的实际研发经验, 当时因为knime 2009年 只支持表格 输入与输出, 不支持object传输, 于是作者设计etl的 unicorn pipe line 主要动机是进行ReinHart 的计算机视觉作业中能将图片像素, object, 和后来作者音频处理也能将 java sound的音频的计算传输. 在函数计算的过程中, 输出就比较唯一了, 即一个object 就能够解释一切, 于是作者最后将 节点 设计成 一种 编码机 的结构, 编码机的意思是 输入可以多种多样如3个input, 但输出仅仅只有一个格式, 具体的编码解码知识点, 作者的编码机知识点来自于印度基督大学 jpsir 的 digital logic 数字逻辑课学习. 描述人 罗瑶光

Implements of the trace of ETL unicorn

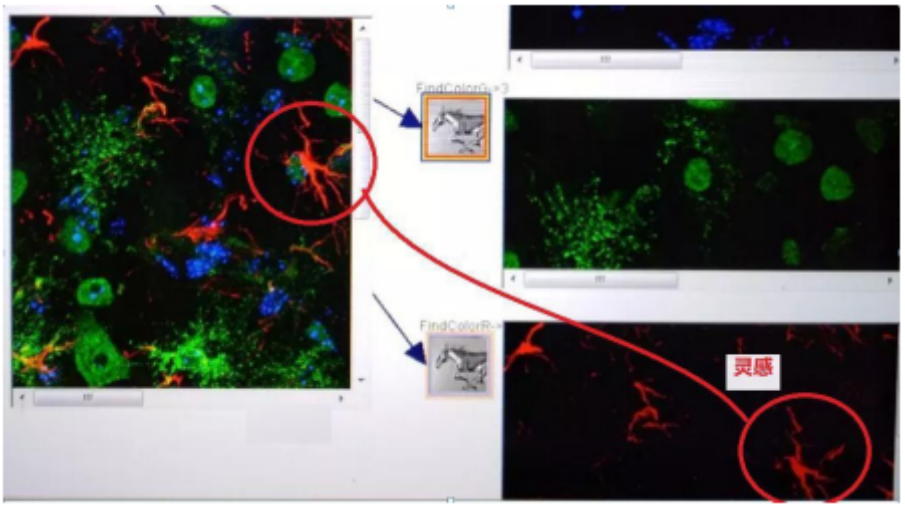
The author started a development of ETL unicorn after he had three years of good experience of Knime SDK, at 2009. The most two reasons for only table I/O format, and non-support of object values, the author needed a ImageIO object to finish his dream of convolutional pipes after <<Computer Vision>> class by Dr. ReinHart. The author had to build a new ETL to support all kinds of input objects. Seem the ETL unicorn did a well demostribution, and now became an engine of Tin-shell nero-pipes.

ETL节点皮肤系统

1 德塔ETL 皮肤采用bitmap实现, 可以自由替换. refer page 333, 334

关于ETL节点的神经元皮肤文字描述

作者在linkedin中阅读了牛津大学的牛顿霍华德教授的几张人类的真实神经元图片, 于是想把自己的ETL 的皮肤设计成这个神经元的样子, 觉得很高大上. 于是用bitmap的矩阵来描绘出神经元的边界.



图中的左边红绿蓝神经组织图片来自牛顿霍华德的linkedin 发布. 作者 节点皮肤设计形状 发散思维联想 借鉴了红色圆圈内的神经元结构.

作者跟随牛先生做一些疾病病人的非盈利技术研究, 如阅读障碍病人的思维模式研究., 循序渐进的思维研究学者模式. 作者和巴黎一大的学生数据处理, 最后研究有一定的成果.

1 作者识别了关于男性和女性的思维方式 平均密度集, 和遇到问题的思维波动方式,

2 阅读障碍主要的体现方式, 如 阅读点和时间面, 阅读频率等细节上, 以及阅读障碍高峰

产生的时候, 思维的混乱程度, 波形抑制程度等.作者现在的阅读能力很大概率得益于经常在研究, 工作和学习中有效地避开这些阅读问题细节. 对牛先生是感激的. 描述人 罗瑶光

Implements of nero-cell-skin of ETL unicorn nodes.

The original nero-cell picture here and the author referred Dr. Newton Howard. At 2015, the author followed Mr. Newton to do a dyslexic analyzing project, and partner with Paris I's student. Finally got 2 development results lists of.

1 The density of Nero computings between women and wen, and Its mean of ratios about Alpha-brain wavelet.

2 The PCA of dyslexia and ICA of each patient, especially in a dyslexia-peak domain of messy mind, lowpass catching and frequent reading. And these theories of almost are changed from the author's thinking when at those times.

(之后我会进行代码的优化优化, 将 bitmap的表达形式 array[][] swap成 array[]+ array[], 这样50*50的界面 含有200个点, 2500次遍历 就变成了200次, 界面刷新时间缩减到原来的十分之一甚至更短. 罗瑶光)

写清楚点, array[10行][10位]= 1 or true 如果坐标矩阵第10行第10位 是像素显示标记1, 那么可以swap成 array[0]=10行, + array[0]=10位, 这样极大的减少计算array的内存buffer占用. 再举例 array[13行][9位]= 1 or true 如果坐标矩阵第13行第9位 是像素显示标记1, 那么可以接下来swap成 array[1]=13行, + array[1]=9位, 方便理解.

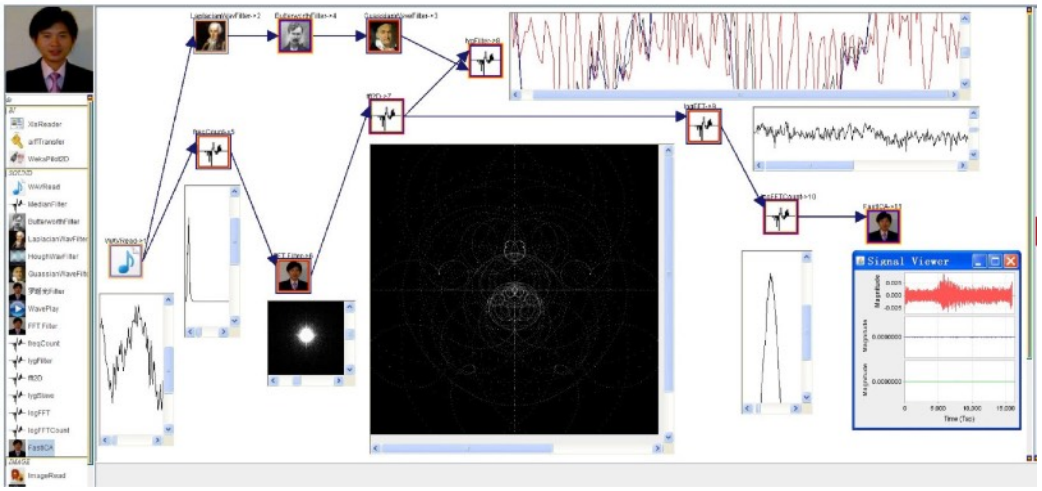
Them has a new method of computing way of bitmap matrix. For example, array[weight][height] was a matrix, made It into an array[n]+ array[n]. So the 200 points of 50 heights* 50 weights of array[50][50], means the ordinary perform were 50* 50= 2500 times, and now was 200 times of array of x[200]+ array of y[200]. The optimization-effect ratio was 2500/200= 12.5. Made a well example of input array[10][10], which equals true, so swapped for array of x[0]= 10 and array of y[0]= 10, then the next input array[13][9], which could swap for array of x[1]= 13 and array of y[1]= 9, It seems fast and cool.

2 德塔ETL 皮肤包含指标, 箭头, 连线, 节点外形, 控间外形设计. refer page 332, 319~

3 德塔ETL 的控件采用jdk的组件 component进行2次开发. refer page 334~

4 德塔ETL 的引擎界面的描点画线基于AWT Swing的canvas 画布系统实现. refer page 311~

另外德塔早期曾用界面 LYG-AI 如下:



ETL 流存储

1 德塔ETL的流存储格式

是一个object类, 可以包含多种状态. refer page 329~332

2 德塔ETL的流中间态存在方式

,采用 单例 的this. clone, 确定了中间态存储模式. refer page 见 节点 clone()

3 德塔ETL的流序

可保存, 可观测, 但不可逆. refer page 329, 330

ETL 节点

1 德塔ETL的节点类比

作为一个计算单元, 模拟神经元作为最小单位计算. refer page 327~

2 德塔ETL的节点UI设计方式

存在UI多样化, 可界面设计如orange, knime, weka那样. refer page 335

3 德塔ETL的节点与语言类比

也可以语言化, 如PLETL的语句IO模式研发设计. refer page 774~, 790 --如tinshell计算节点流

4 德塔ETL的节点IO模式

大体为从左到右的IO模式, 节点流计算目前可循环但不可逆. refer page 329~332

ETL节点类OSGI插件

Gitee 20190626 感想: 关于作者 ETL unicorn 的 OSGI 思想灵感动机来自 liferay 的 theme 控件, 关于liferay 的启发作者有必要描述下: 2012 年, 作者在路德大学学习网络课程用liferay来做, 当时看见liferay的皮肤比较漂亮, 心想如果自己能做个视觉软件也能有皮肤多好. 作者在网络课上专门做了个 presentation 关于 liferay theme做视频集成的文档架构. 作者欣赏spring的整体性, 在这里给个大大的赞. 作者当年美国上班, 有3年时间都和spring 的应用打交道. 作者想到这里, 2012年准备自己做个网站, 于是在加州路德大学的图书馆借了一本《cgi 网站建设 C语言基础》, 并模拟了c做所有HTTP, web, cgi, socket, tcp, ssh, router等--路德图书馆有我借阅记录.

网站网络通信原理实现. 在阅读和学习这些各种各样的网络通讯协议中作者对环, 令牌码, 字节, 套接字, 掩码, 波特率, 压缩, 冗余检测方式有了综合整体的系统观, 发现对象继承的重要性(000~888之间的各种PROTOCOL协议不都是一层层的继承么), 那些独立节点只要继承这个注册属性, 不就自适应了么. 于是我就那样做了.

Implements of OSGI Notes on Gitee 2019-06-26

The author's inspiration behind OSGI could trace the liferay-theme applications and presentations at class of <Adv Computer Network>, 2012. Then the development of ETL unicorn. The author thought that canvas, which needed Its own theme-skins such likes Liferay. Then tried to build unicorn-theme-UI by using original JDKs. And now became an UI part of YangLiaoJing software.

The author appreciated the spring and its systematically. Since 2012, the author simulated CGI webs by using C at Callutherans, Its emphasis on HTTP, Web, CGI, Socket, TCP, SSH and Router. He considered an object oriented of web protocols of 000~888, which could be used in an optimization-inheritance of socket registrations. And now became a procedure part of VPCS schedules.

1 德塔ETL的插件开发模式

类似OSGI的jar开发模式. 严谨地说只是继承的classloader模式. refer page 286, 290, 777

2 德塔ETL的插件的识别方式

可以加元基标识认证组件, 避免错误插件扩展. refer page 290, 777, 779

3 德塔ETL插件目前应用支持

插件进行平台配置, 页面扩展和节点扩展. refer page 286, 290

4 德塔ETL插件名元基索引目前应用方式

的3元基文件名索引肽化支持插件的分类管理和加密标识. refer page 781

关于作者插件思维溯源描述.

作者2009年第一次接触OSGI的思维来自Knime SDK 节点插件设计, 但一直没有时间进行跟进节点插件编码领域的研究, 因为作者早年迫于生计, 一直在互联网商业公司和Spring MVC + MySQL 架构体系打交道. 2012年作者第一次思考OSGI的模型是在加州路德大学实验室, 作者在写计算网络课 香港教授的 web架构论文的时候, 有用到Liferay小窗口体系, 觉得这是个趋势, 作者开始下 Mini OSGI 关于Camel的WEB 部署用法, 因为API是现成封装的, 作者部署了几个RPC Demo 之后就莫名毕业了, 没有再跟进研究. 2013年作者设计完 ETL Unicorn, 没有采用OSGI技术, 仅仅是继承实现同类接口节点扩展 基于override 和overloader. 2017年 作者面试Infosys 持续长达2个月, 被面试官电话频繁问关于classloader的技术, 作者准备了标准答案关于Java jni native调用C/C++ 的 Object Scan 技术原理, 面试官说OK, next question.. . 2019年之后, 作者有时间开始用classloader 将2013年的ETL Unicorn继承方式设计成插件方式(所以作者的节点设计一直不在作者ETL著作权内. 之后作者采用Tin-shell 新作品统一化设计节点.). 目前作者的插件接口开始三元pdc语义编码索引插件文件名肽化标识, 用于之后的元基新陈代谢应用进化探索. 描述人 罗瑶光

Implements of the trace of unicorn's plugins.

The author first times to touch the OSGI, does since he the first time to develop the Knime SDK, 2009. Then touched the Camel and mini OSGI rpc applications of 2012, He had been working as backend developer by using Spring+ Mysql during 2014~2017. The inheritance of ETL node classes were simple like the template-modes of overrides and overloaders during 2013~2014. After the 2 month-duration and exceed 30 times of Infosys interviews and frequently be asked of how about the classloader, Object Scan and JNI native parts by new JDKs, the author thought the ETL needed a change of Its node extensions by YangLiaoJing integrated. And now these classloader-plugin file-names became a 3 metas-PDC (DETA sixteen meta base, INITON char-code) word branches of DNA indexing and encoding.

感想Gitee 20200427, 关于osgi功能的注释: 一开始作者不想做成插件方式的主要原因是这个ETL极其轻巧, 主要针对嵌入式系统, 插件的过多会增大系统整体体积, 后来在设计华瑞集的时候, 作者理解了一个思想: 万物皆为插件, 于是觉得, 以后过度到第二代电子医院系统, 作者会将所有功能全部进行插件化, 因为华瑞集不是嵌入式系统, 严格的说它是一个巨系统, OSGI思想在这里用到非常合适, 于是没有多想, 就做了.

Implements of the theory of 'Universe Inherit/万物皆为插件', Notes on Gitee 2020-04-27.

Initially, the author built an ETL unicorn for where be used in embedded system. In order to make the ETL to small and smart, therefore he didn't use the OSGI plugin theory to make a widely extension and only an ETL kernel instead. And now became an OSGI extension parts of YangLiaoJing software. Seem the OSGI extensions could fit a big system of YangLiaoJing. The author considers an OSGI extension does more better than object-oriented inheritance in this situation.

ETL档案存储方式

1 德塔ETL的档案包含节点流信息和节点配置信息.

refer page 279, 282

2 德塔ETL的存储采用节点的画布状态单例信息存储方式.

refer page, 279,, 282

3 单例信息包含

画布中节点的坐标, 名称, ID, 连线, 配置信息等实体信息. refer page 282

4 德塔ETL的流存储文件

用文件读写形式. etl后缀存储. refer page 282

5 德塔ETL的存储加密方式

可支持加密和batch模式运行. refer page 养疗经应用略

Deta的 ETL 加密采用自主研发的元基加密方式. 作者20220403认为这些一个个档案便是一个个神经组织区间.

ETL的拓扑流方式

1 德塔ETL的拓扑应用点

体现在节点的神经元模拟计算观测. refer page 273

2 节点的神经元拓扑方式

模拟拓扑体现在从左到右的从高到低拓扑模式. refer page 273

3 拓扑缺陷

第2点或许是个拓扑缺陷, 但是却因此又确定了固有 的向量方位. refer page 273

关于拓扑的描述

1一对一拓扑作者之后的TSP极速商旅坐标欧拉路径计算的早期版本也是通过拓扑循环计算优化而来的. 2一对多拓扑作者认为是组合数学的基础. 用真实的例子举例, 作者的DNA概率钥匙加密体系便是一种概率组合数学的应用. 3 拓扑是神经网络的基础. 在计算机领域, 拓扑学的价值不菲, 用真实的例子举例, 作者的ETL节点链接是一种计算机图形学上的UI拓扑结构. 4 作者的第六章数据预测 有力的证明坐标团非线性计算 充斥这拓扑学术的影子. 举例如 团中心轨迹链接和趋势计算, 以及切裂团的向量压力标识. 总结, 因为这种1 欧拉拓扑与 2 组合拓扑的思维, 量变催化出作者研发出极速高阶欧拉融聚商旅团TSP路径算法. 另外欧拉路径拓扑与邻接矩阵拓扑变换, 量变催化为作者的十六进制元基排列打下了基础. 描述人 罗瑶光

Implements of topology.

Extend it one per one for the DETA Ouler TSP, and Its ordinary editions, were based on this discrete and topological combinations. More examples of one per many topological combinations, were DETA DNA encrypted system, and PDE probabilistic key. The author considered the topo's was a fundament of Nero-network. For examples of ETL unicorn pipe UI, DETA unlined Ouler TSP, and trace the central weights of each fissile group. Now became a powerful proof in Ouler TSP and adjacent way of DETA hexadecimal meta base INITON.

ETL 模拟神经网络

1 德塔ETL的神经网络实际价值

计算在流数据计算中有实际价值. refer page 274

2 德塔ETL神经网络学术价值

在PLETL和Tin-shell中逐渐体现其更多学术价值. refer page 783

3 德塔ETL的向量拓扑模式

确定了神经网络的加权方式. refer page 274

4 德塔ETL的神经网络流序模型

可循环不可逆. refer page 274

ETL 节点流一键执行

1 德塔ETL支持 一键保存.

refer page 277, 282 --早期ETL 保存的数据是关于其节点的配置内容, 通过文件的行写形式来保存, 现在Tin-shell的节点已经统一为语言的模型, 于是行写逻辑得到了简化, 方便和提高了元基加密和解密的准确度。

2 德塔ETL支持 一键读取.

refer page 302

3 德塔ETL支持 一键执行.

refer page 308

4 德塔ETL支持 一键清空.

德塔的一键执行最早作者想设计成Knime的那种batch的批处理脚本节点流那样, 随着作者Tin-shell出来后, 作者觉得批处理跨语言, 既繁琐又没效率. 于是想做成JAVA脚本语言, 伴随这个思路, 目前作者设计了Tin-shell语言和元基花语言.

At the first time, the author tried to build a batch script to do the time schedules such as Knime. But is seeming did lower effectives and much messier computings by using multi-languages (mixed of Java, C, Shell, or others like Python and Ruby

etc). Once the Tin-shell done, seem could be unified and conducted. Because it all based on Java. Now became an evolutionary-language script of DNA INITON-dandelion.

Gitee 20190618感想, 刚把这个一键档案执行的 线程版本进行了完善下, 我是这个思考的, 将线程进行单例类执行方便运维分类, 同时我还能进行OSGI 思想继承运行不同种类的档案. 每一个档案作为一个线程, 这个线程我定义为神经流线程, 线程的全局对象引用, 非常方便多个神经流蜂群计算形成神经组织. 也是海量并发计算的基础, 我现在思考的方法是每一种 特定神经组织 一个 skivvy管理, VPCS管理层过多, 每一个线程作为一个sleeper服务对象, 我是这样想的, 我觉得有必要这样做, 现在的条件能做, 我动手了. 我要做个set类出来, 把线程处理的数据载体分离开, 不能因为线程死了, 运算的中间态数据就直接仍垃圾桶了. 高效节能环保是主题.

Notes on Gitee 2019-06-18, Once done the 'one-button' of document-execution. The author considered the pipe execution of each document where could be a single nero-cell's organization-flow. And these documents or flows, could be arranged by VPCS-schedule of skivvy management of memories, hallkeeper registations of processes, sleeper performs of schedules, and dream sets of garbages collection. Means a higher efficiency of the environmental protection about energetic computing-optimizations.



关于 PLETL 与 ETL Unicorn函数分类的 文字描述

一开始作者把ETL 的节点操作设计成普遍的UI界面操作模式, 这个问题来自于作者2008年发现knime的界面智能用其自身的SDK或者SWT来实现. 作者设计了ETL Unicorn, 便直接Swing+AWT来跟进实现界面即可, 永久告别第三方包, 自从Tin-shell的脚本语言项目搭建诞生, 作者开始用文字命令来实现节点功能操作, 认为所有的节点可以统一成一种界面结构, 即输入部分和输出部分, 输入脚本命令, 输出结果. 于是节点就被格式化简便了, 减少了美工研发时间. ETL 引擎函数的维护比较稳定. 从某种意义上来说, Tin-shell的出现, 永久告别了 市面千奇百怪的节点设计. 节点框架开始走向统一.
描述人 罗瑶光

Implements of ETL Unicorn and PLETL nodes.

At the beginning, the author made an ETL node which almost similar to the Knime node, because the Knime node is a standard ETL node (with orange, WEKA, oracle, kettle and etc are almost the same) which contains an operating UI. Based on this self UI, the user could do more interactions between ETL nodes and assignments. for example, Knime user uses the SWT technology to build all kinds of the procedure UI by using the Knime of eclipse plugin SDK. Once the Tin-shell script was build, the author thought It could be integrated in the ETL work flows such as PLETL, thus, each Tin-shell ETL could only have had one type of procedure UI, which means Tin-shell UI-- A top Text input for scripting Tin-shell commands, and a bottom Text output for responding results of Tin-shell executions. The author once said: all of ETL node UI could be the same type as Tin-shell node in the future AI.

于是开始思考，难度定义在专科和本科3年以内，

1 ETL的扩展应用方面，比如神经网络的拓扑快速计算方式

2 ETL的演示应用 如mind 和 VISO 等应用，如何快速自适应变形，

3 ETL的各种动态皮肤 视觉特效 如何插件化设计

4 ETL的分差流如何让批处理

5 ETL的可视化界面如何脱离JDK 浏览器执行，如 Jflash， JOCX，Jscript， 非java控件封装。

6 手机界面如何布局ETL，

第四章 德塔 Socket 流可编程数据库语言引擎系统

基础应用: 元基催化与肽计算 编译机的计算存储机

知识来源,

作者第一次接触数据知识,是在本科的数据库课上,萨师煊,王珊的绿皮书,非常系统地接触了数据库范式,数据函数,和数据库 query. DDL, DCL, DML的定义. 这里表示感谢.

作者在印度基督大学的 数据库管理课程中,第一次专业级地接触数据库的格式定义,如tupe,cell,row,spect,table等,搭配,范式优化,对象存储寻址方式,在cobol74的实验课有涉及 文件表的cobol、管理方式作业,以及作者在c语言设计作业有高分完成文件的读写实验. 关于web VPCS调度,作者要感谢印度基督大学的操作系统课程,作者的基础来自这里,作者的mvc, mvp的基础来自作者在章鑫杰那设计蓝牙广告机的设计经验,作者用谷歌搜索了大量的mvc和mvp的国际论文资料阅读.

作者在上海章总那,章总有让作者在赖静旁边看他展示spring MVC+hibernate Many-Per-Many reflection+struts 1个小时观摩,这是作者第一次接触hibernate的应用. 另外作者感谢赖静在作者设计knime节点的时候,帮作者解决了很多基于alfresco的编码基础问题.

作者之后去美国路德大学,广泛地接触了数据库应用,oracle OLAP,高级数据库系统,制表管理,数据库语言,以及web数据库SAX,JQUERY, Xpath, XQUERY,的实验和作业和 query优化,内存表优化,计算指令优化. 有了这些基础,作者在美国上班5年一直是个web后端程序员的职务,普遍基于java和php的语言进行设计后端的rest数据库应用业务,基于jboss, resteasy, springMVC, springboot, laravel, mybatis, ibatis, hibernate JPA, criteria, mysql, mssql.

2009年作者有对湖南星沙国土局的招聘官说作者能独立设计数据库,2019年作者思考了下不能食言,吹了牛就要认命,于是有魄力开始设计了基于VPCS调度的握手web plsql数据库语言,现在发现收获颇丰,由衷的表示感谢.

罗瑶光

Socket rest TCP握手协议

1 协议配置方式

德塔数据库的 admin界面采用 web页进行配置操作. refer page 376

2 协议访问方式

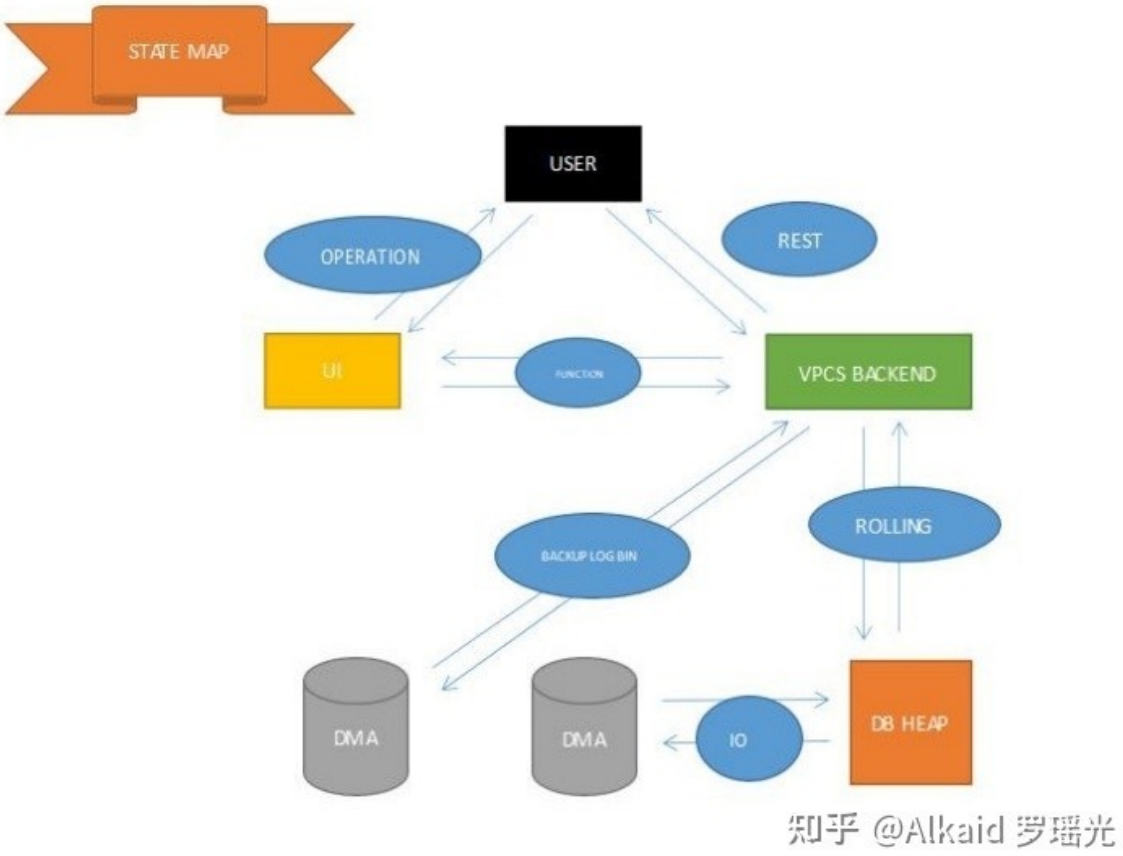
web页配置操作采用TCP握手访问模式, 基于socket的http请求握手. refer page 464~

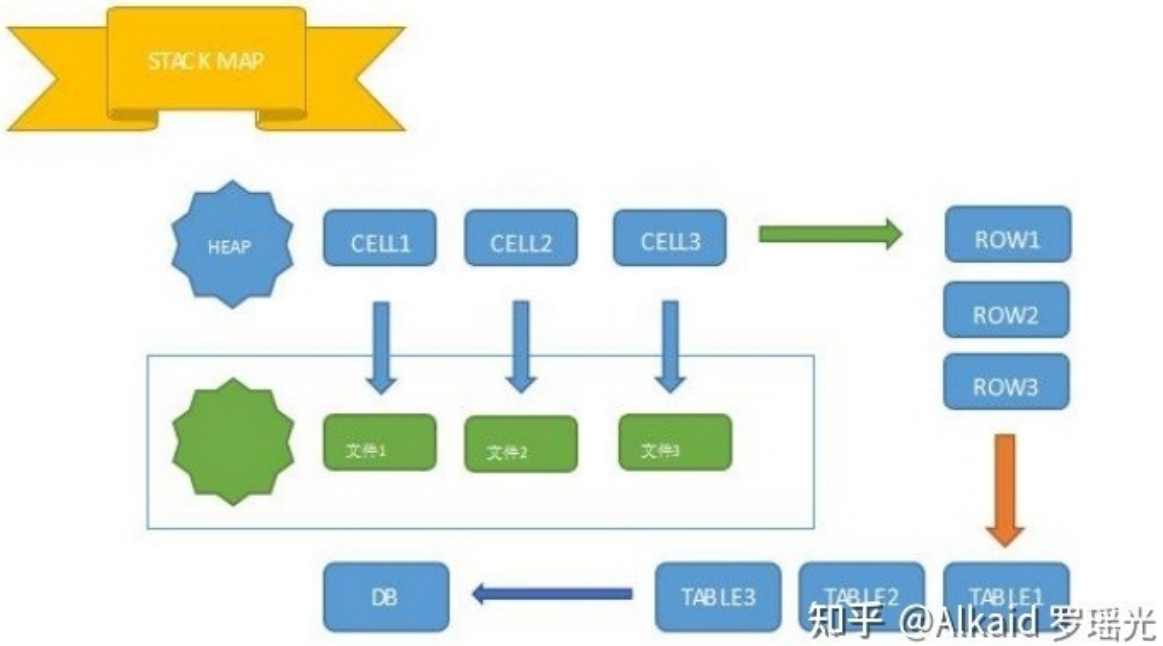
3 协议封装方式

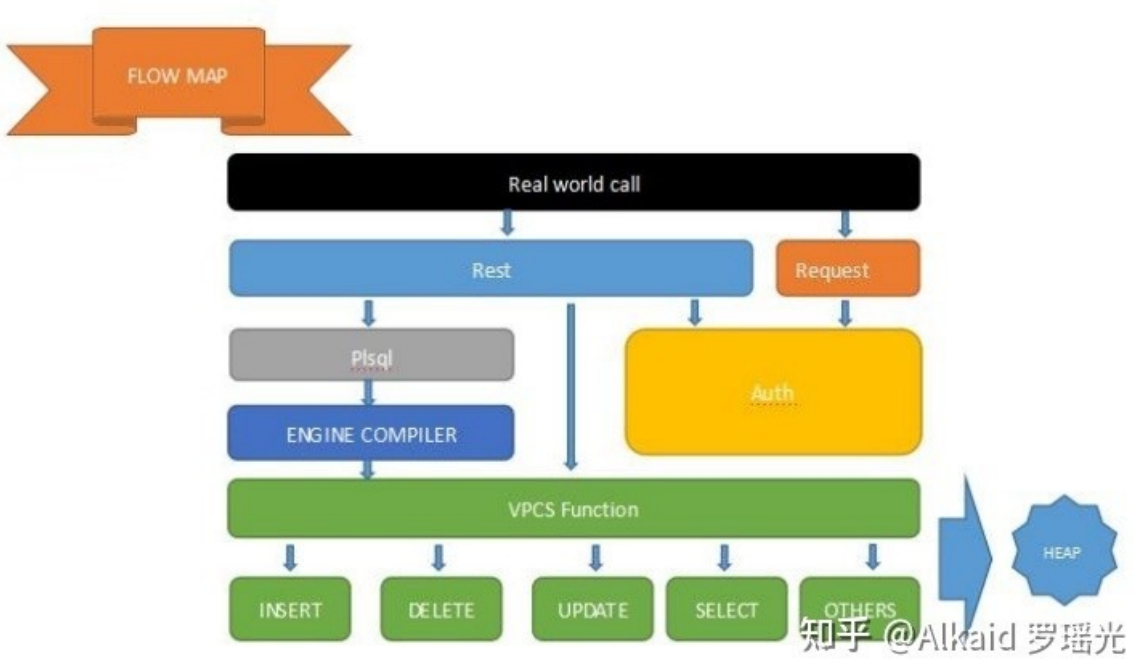
德塔数据库将socket握手进行线程封装, 然后多线程组织页面. refer page 392,

4 协议admin管理方式

封装和组织页面设计过程逐步进行优化形成VPCS后端管理体系. refer page 383, 476


STATE MAP
USER
OPERATION
REST
UI
FUNCTION
VPCS BACKEND
ROLLING
BACKUP LOG BIN
DMA
DMA
IO
DB HEAP
知乎 @Alkaid 罗瑶光


STACK MAP
HEAP
CELL1
CELL2
CELL3
ROW1
ROW2
ROW3
文件1
文件2
文件3
DB
TABLE3
TABLE2
TABLE1
知乎 @Alkaid 罗瑶光



文件数据库

1 文件数据存储方式

德塔数据库的数据存储是一种文件存储模式. refer page 408, 409, 469, 473

2 文件数据计算范式

文件的读写进行子集, 行, 表, 映射, 表头, 按1范式分类. refer page 375, 434,

3 文件数据加密方式

数据库的数据读写支持加密. refer page 见元基加密

4 文件数据堆栈方式

每一个文件不但有物理空间, 还有相应的内存空间. refer page 375

VPCS服务器

Gitee 20190620 感想: 关于VPCS的描述: 最早我的项目都是基于大众化MVC结构, 这个结构陪伴我5年的时间. 第一次使用mvp是我在设计ios的暗夜法师游戏设计时候要进行对象数据小规模刷新机制, mvc太重了, 我开始将CONTROLLER 细化. 我的结构开始走向mvpc模式. 当时感觉是不错的, 后来在用spring mvc的时候, 普遍是IOC的mvpc设计进行自动注册封装, 一次我在进行mock mvc测试的时候 偶尔一个细节打动了我, 就是我看到的并发是个假象, 只是个list的先行注册分解而已, 我后来在用spring boot, thinkphp, laravel, zend 等框架的技术, 我这个问题重复了100遍在脑海里, 是什么封装驱动这个注册分解模式? 于是我开始用c moonguse来做线性服务器, 并模拟了, rust, golang等编译型注册原理, 我找到了实质, 那就是tcp握手规则, 我开始设计tcp rxtx进行数据交换, 设计了VPC结构, 发现一个大问题就是数据没有格式化, 于是我开始寻求一种在多线程线性服务器运算握手的时候能够带数据交换的一种常用格式(稳定的)的进行业务, 多好? 我找到了就是socket 套接字. 于是我将socket进行 服务器部署, 将文件数据,

脚本数据, rest接口数据, 动态数据, 等进行一个个函数封装, 然后将函数的数据和socket线程分离, 便于运维, 测试, 数据计算浪费管理, 这就是VPCS第一版本. 特点就是数据与逻辑分离, 线程与资源分离, 业务与控制分离, 之后蜂群计算阵列, 非常方便我的INITON DNA 催化布局. VPCS第二个版本是我在尝试用springboot mybatis 布局的时候, 发现局部注册太臃肿了, ***主观情绪略***, 于是我开始设计第三代VPCS版本, 将v p c s 四个部分进行全面的map注册模式, 我当时想到的好处是 1 以后map reduce 非常方便, 2 全部量子碎片化, 速度性能要爆炸, 3碎片逻辑进行微分催化1对1 INITON mask进行肽链重用, dna的早期框架我有足够条件进行论证, 现在VPCS进入第4代了, 我开始考虑情感化线程操作. 我一直在准备着. 让情感与计算分离.

Implements of VPCS server notes on Gitee, 2019-06-20.

In an early time of author's projects, MVC backend was a popular solution for web development. And the first MVP project of IOS game:

https://github.com/yaoguangluo/dark-wizard-game/tree/master/DOW2

, the author appreciated Its object oriented schedules and Its refreshment. Due to the MVC was too heavy, so the author did a definition of game controllers. And finally separated the macros and prototypes as a type of MVPC. Since the author did a Java test job at Folsom Intel, after he used Java mockito MVC API, mixed with Junit4 to make a springboot and hibernate test. He found that all functions could be tested as a single process, and this process contained a classify of schedules. These schedules included singletons, sonar spells, spring web boots and spring injects. And these schedules were tested one by one like a time sequenced line-waterflows. So, the test process before deployment which was a synchronization, but the true of Its inner schedules were an a-synchronization. The author took more questions about sonar qube registrations because those contained lots unforeseeably encrypt dlls. The author tried to explain well between a-synchronization and synchronization by reading the professional book of Operating System, TCP/IP, Software Engineering and How to programming. After he did a simulation by using CGI, Mongoose C, Rust and Golang etc. Then he got a result of TCP hands shaking and time sequenced Rest-calling. He became to know the a-synchronization distinct to synchronization. Then announced a first 'VPC' at

https://github.com/yaoguangluo/VPC5.0/commit/dc06dbc3750879155871d45ce9cdb8be29825a60

https://github.com/yaoguangluo/document/commit/390b77a854ddcba2fe38021b184d2dae5e22975f

2017年10月19日 GMT+8 12:15. Because of his VPC project, was built by springboot and mybatis, which was unforeseeably. The author couldn't test and catch the question well, then try to code a new bottom OSI layer of rest call allocations about his theory of VPC. He began to think about camel RPC, Golang grpc, TCP RXTX(NIO) hands shaking, RS485, Powerlink flips and Modbus hands shaking. Finally, the author used SSH Sockets to simulate the hands shaking CGI. After he had simulated an iluwatar handler-schedules, he got so many efficiency problems of memory garbage, unused performs and dead locks. Then he try to make separations of data and logics, schedules and resource, business and controllers. And now the VPC became a VPCS, VECS of AOPM VECS IDUQ TXHF, a part of DETA DNA hexadecimal meta base INITON. For the listed bee coords classify of 'Swarm Computing' at chapter 6 and DNA AOPM-VPCS catalytic computing-array at chapter 7. The author used socket flows of hands shaking, to instead springboot and mybatis, the author did an affordable DETA PLSQL, to instead MYSQL. And now he prepared to make a new separation of ethics and computings.

Gitee 20190625 感想: 我花时间进行论证VPCS 和 rpc, mvc, mvp, 微软的handler event 优势和差异性对比.

VPCS定义: 是标准的从MVC(mode-view-contoller)到MVP(mode-view-process)到MVPC(mode-view-process-controller)到VPCS(view-process-controller interface- static map)的一个过程. vpc 在第5版本时候, 作者将mode去掉了, 将 process分成 sleeper 和 sets, 正式命名VPCS

第二版VPCS 作者将view 拆分为 vision 和 pillow, 逐渐形成 vision- pillow-controller-sleeper 的 VPCS 结构 第三版VPCS 作者将 set 保存在pillow中 让sleeper线程和pillow分开 让skivvy来管理, 逐渐形成 vision-sleepers- pillow-controller-skivvy 的vspcs结构 做到逻辑与任务分开, 线程与资源分开的模式.

VPCS随着工程的发展已经进化到了 HVPCS (hallkeeper-vision-pillow-controller-sleeper) (运行与维护分离, 执行与逻辑分离, 任务与资源分离) ***主观情绪略***, Hall的词汇来自作者在路德认识 留学生服务的Hall教授.

1 对比微软的handler events 优势: vspcs更实用于让大型综合调度瞬间简化, 因为本身就包含了RR机制思想. 2 对比web的rpc 优势: rpc大家可以看下illuwatar的包 和 go的rpc 异步消息机制, vspcs的机制是通过触发来运行逻辑http 协议是socket流进行rest, bits等处理(类似的功能软件是redis), 3 而rpc是 tcp连接当然连接速度会更快, refer go 的最新 grpc https://blog. csdn. net/lk2684753/article/details/84436190 ***主观情绪略***. 4 illuwatar rpc 是微软c#的handler events 线程注册机制的java 注册封装模式, 是时时动态的, 计算量增大. 5 vspcs 采用酒店的服务管理思想进行程序化, (***主观情绪略***)这是世界第一篇关于酒店业务调度思想的软件论文. 作者设计和更新它的需求的直接动机是为了方便日后进行海量并发的VPCS矩阵蜂群计算中满足 低计算量, 低浪费量, 无监控死角的微分催化计算业务.

作者再加点addition: Apache Camel, 作者在几年前用这个包做异步消息, ***主观情绪略***, 作者refer 2本书 1 java. net 白皮书 2 java how to program 6th 蓝皮书 开始用socket做握手发送数据包, 发现速度不错. 再加点料, 为什么作者会接触camel, 作者在加州路德大学读书的时候有一次在计算科学实验室做嵌入式OSGI思想研究的时候, 在谷歌上推荐弹出了个camel, 这是作者第一次看到它. 关于作者ETL unicorn的 OSGI思想 灵感动机来自 liferay的theme控件, 不是camel, 在这里申明下, 因为作者的动机是要走节点图形的继承, 而不是插件化OSGI包. 关于作者的 ETL unicorn的 OSGI 函数更严格来讲不是OSGI, 而是继承的子类对象们进行统一分类管理而已.

Implements of VPCS server notes on Gitee. 2019-06-25

The author did a contrasted analyst of VPCS, RPC, MVC, MVP, and compared each to Microsoft's Handler and Event. He did a conclusion of VPCS, from MVC(Mode-View-Contoller) to MVP(Mode-View-Process), then again from an MVPC(Mode-View-Process-Controller) to VPCS (View-Process-Controller Interface- Static Map). At VPC5.0 edition, the author had removed the Mode, and separated Mode and Process into sleeper and sets, announced It as VPCS.

Then The author did an evolution of VPCS, to separate the View into Vison and Pillow, gradually into a different type of VPCS (Vision- Pillow-Controller-Sleeper). Seem not done, the author continued to integrate the Set into Pillow, and built a skivvy to arrage the Pillow set. As a VSPCS(Vision-Sleepers-Pillow-Controller-Skivvy), and now became an HVPCS(Hallkeeper-Vision-Pillow-Controller-Skivvy) by instead Skivvy of a Hall. For the VPCS HTTP server and schedure. (Not for DNA INITON Encoding, which was a simple of Vision, Execution, Controller and Static).

Compare to the Microsoft handler events, VSPCS schedule and Its RR (Round Robin or Early Birds) could be simpler than other factory modes. Compare to the RPC, VSPCS simulated the concurrented, asyntronized, inner time sequenced threads-schedule (see HTTP request and responce Java sources), instead of illuwatars incremented, asyntronized handler and event. Golang RPC refers (https://blog. csdn. net/lk2684753/article/details/84436190). VSPCS did an example of Hotel-Daily-Arrangement. Because of 3 messy environments as Hotel-Daily-Arrangement, Hospital-Daily-Arrangement, and Bus-Stations-Daily-Arrangement. The author considered It could do well in a business of catalytic computings. In order to satisfy a reduced computings and wastes, and raised a seamlessly monitoring. The author thanked Operating System, Apache Camel and Its Mini OSGI, Java.net, Java How to Program 6th, Liferay and Its theme system here the Cheers.

Gitee 20190826 感想 为什么我不用spring?

很多年前, 很多人带着各种各样的问题问我为什么Java spring 那么好了, 为什么 我要另创socket流分发服务器? 我可以告诉大家的动机.

1: 安全和同源配置问题. 2016年我在美国注册了comforx软件公司, 当时设计个电商系统, 因为我用的是spring后端, 我发现一个大问题, 为什么被同源限制, 如果是拒绝恶意攻击, 至少可以用户自定义设置配置呀, 而这个功能一直被spring给控制了, 当时我郁闷的用2台电脑和2个不同的公网域名IP才解决的这个问题. 我的动机很明确, 我要自己写web服务器.

2: 市场的开源产品大, 重. spring 我觉得太大了. 2015年我在亚米上班的时候竟然听见吴迪和little说java太重了(little想用C), 太重了, 内存, 什么的太大了, 我当时候在想, 可以用spring boot 或者golang 呀, 我当时给王浩雷推荐了go lang, 王浩雷说 go lang 当时生态系统不全, 做工程没有java 安全和高效, 特别是开发周期.

3: 方便我的VPCS实验论证. 最早接触socket是在印度那本java how to program 6th, 第一次使用socket 是在上海章总那, 做 C# 蓝牙 大文件传输, 用到了32feet, 当时我问章总, 为什么我的软件智能一台一台手机单独发送, 不能群发, 章总给我一个别人的C#软件(2012年那个硬盘我在美国Del Ma AVE "弄丢***主观情绪略***"了, 记得QQ(313699483)里面有我和爸妈聊天的记录), 说这个别人实现了你看看多线程的原理, 我说多线程我知道, 于是我专门独自写了个多线程哲学家进餐算法, 章总当时之后没说话, 我后来(2014年)意识了, 不是多线程的问题, 是我的握手没有并发管理. 这个socket并发握手, 我特意做了个sleeper Hall 来管理. 以及后来的 VPCS论文.

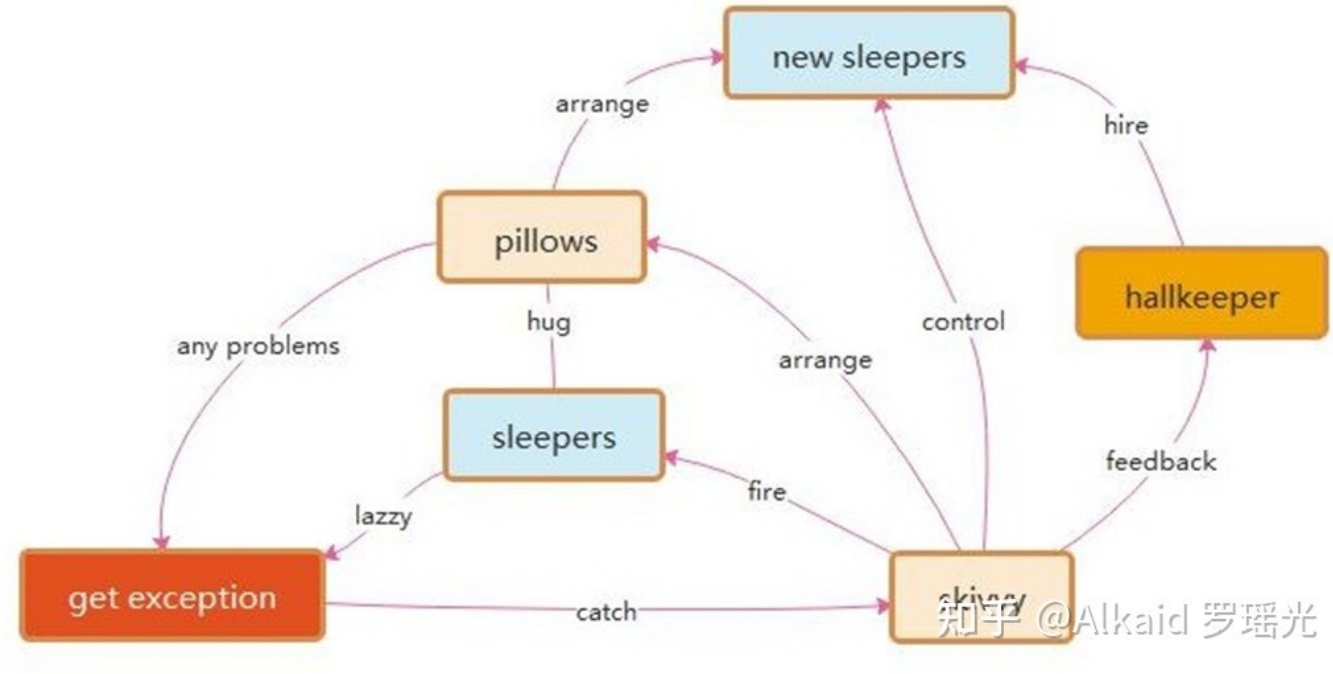
4: 统一, 全局, 生态化, 高效. 小到极致, 我的数据库不仅用socket流, 我的缓存, 前端, 后端都是采用我的socket流, 现在我一台电脑启动整个前端, 后端, 缓存, 和数据库, 4个工程, 2兆内存, 全部搞定.

5: http协议成熟. 为什么我不用rxtx, 我之前已经解释了, 因为rxtx的数据还是原始数据, 协议太低了, 对我的工程设计研发周期变大, 所以我采用http socket流模式.

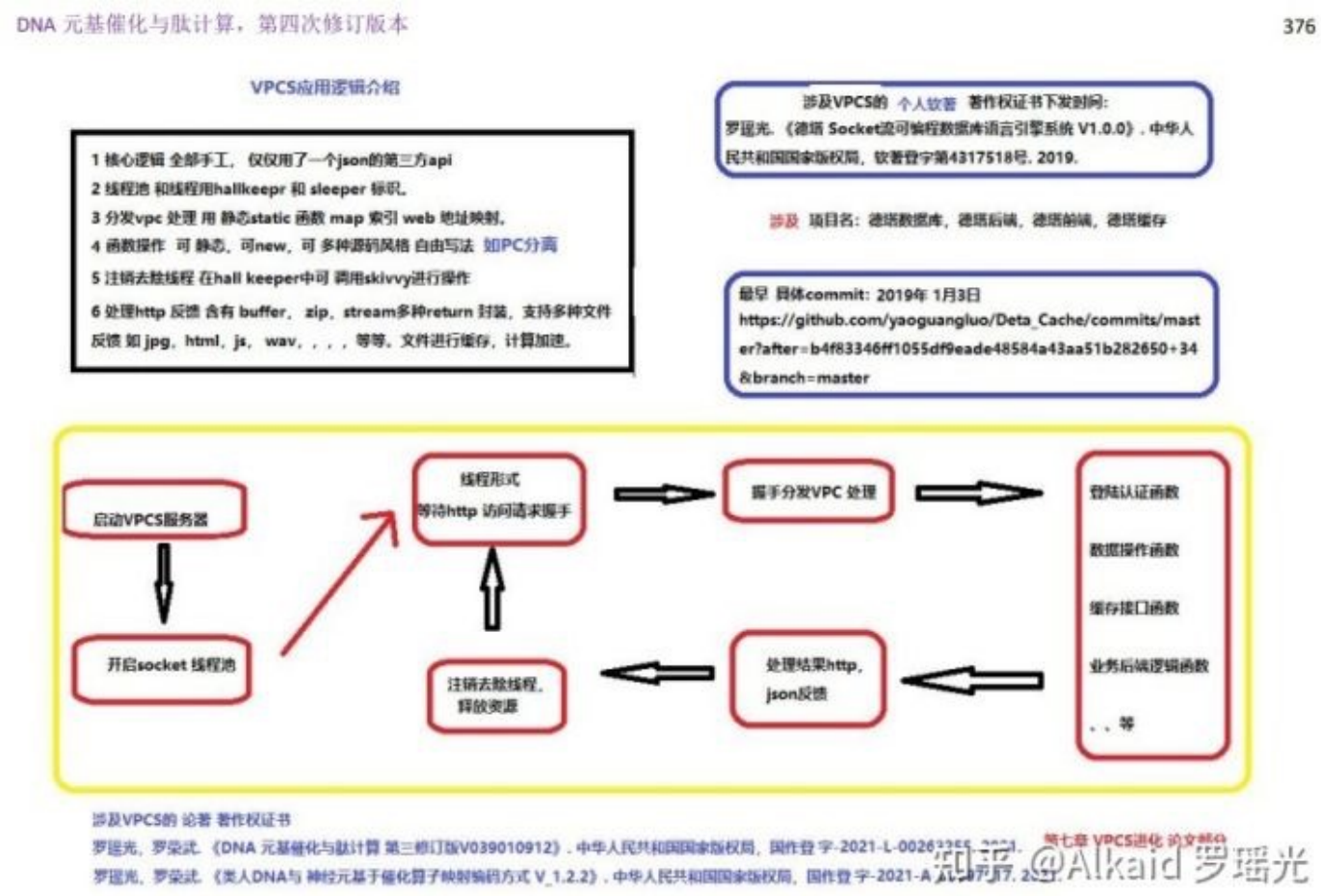
之前我还遇到了些问题, 比如jar, zip, wav, mp4, js, 等文件怎么分发? rest是最好分发的, 因为就是http而已, 可以js 就不一样了, 有 application js, 还有file js, 记得几个月前我一直苦恼 application js的分发, 后来我设计了 unzipped buffer 等格式, 针对不同的web 对象数据全部搞定的时候, 我发现我的socket服务器源码 核心代码 40kb都不到, 哪有spring 100兆, spring mvc 17兆, spring boot 40兆等那么大, 关于我的socket基础怎么来的? 感谢三本书 3堂课: 《java how to program 6th 蓝皮子印度买的》《java net白皮子 美国工业城市frys买的》《c cgi 路德图书馆 》/ 路德大学的计算机网络, 本科的通信原理, 基督大学的数据结构, 操作系统. ***主观情绪略***

还有, 关于一些零散的记忆累计, 我也需要感激的:
1在chinacache的时候管理互联网ip认证项目, 我非常系统地接触了tcp握手的协议码解释.
2在路德大学上香港老师的计算机网络数据编码课非常系统地接触较验机制和TCP编码原理.
3我在2个大学中系统的学习了2次操作系统专业课程.
4我在上海电气接触过power link的tcp 以太帧.
5感谢w3c, web2. 0, http协议官网给我提供了常用文件的 header头 定义规范.
6分发压缩我要感谢gzip, 我的socket数据库引擎系统个人软著不包括gzip压缩, 说到压缩, 用处是真的很大. 因为它让我的握手时间和效率大大增加了.

最早我没有用到压缩的, 因为服务器是花生壳转发, 所以我握手比直接服务器发布慢了一点, 而且走别人的分发代理, 格式还要被强制统一, 我前几个月为了这个问题头疼了几天, 后来用gzip统一压缩大文件, 10兆变2兆, 格式保障了, 速度也上来了. 小伙伴如果不想用gzip, 可以剔除掉, 记得把forwadtype里面分流的函数修改下就行了.



VPCS调度内核



1 VPCS服务器并发介绍

支持每秒400万QPS的web请求. refer page 389,

2 VPCS服务器通信协议

采用TCP rest request模式, 标准化http response. refer page 388, 395

3 VPCS服务器应用模式

可自由设计前端和后端集成. refer page 见德塔官网 和 养料经admin 两个实例

4 VPCS服务器通信模式

完全支持post 个 get 2种请求模式, 可扩展. refer page 481, 488

关于VPCS应用逻辑介绍文字描述

1 作者使用Json api进行数据的载体封装. 2 线程池和线程用hallkeeper和sleeper标识. 3 分发的VPC forward 处理用静态map, 目的是索引web的http IP endpoint地址映射. 4函数操作可静态, 可new, 使用者可多种源码风格自由扩展. 5 注销和运维机制的线程可以通过hallkeeper来分配 skivvy来管理. 6 处理http的response反馈含有buffer, zip, stream等多种格式封装, 支持W3C 定义的格式文件反馈, 如就jpg, html, js, wav, 这些文件打开一次便缓存记录, 方便计算加速.

VPCS的执行流程为 1启动VPCS服务器, 2然后开启socket线程池, 线程的形式为socket accept进行等待http访问请求握手, 一旦握手成功便进行forward分发VPC 执行功能函数处理, 如 登陆认证函数, 数据操作函数, 缓存接口函数, 业务后端逻辑函数等等. 执行结果进行Json封装然后基于http协议 feedback 和response反馈. 最后使用完的socket 线程进行注销, 释放计算内存资源. 因为socket是一个线程, 所以这整个过程和socket accept 是异步消息模式, 互不干扰, 于是socket accpet一直在循环等待http的握手请求. 描述人 罗瑶光

An implementation of VPCS application. The author uses JSON API to return a result. Then uses 'Hallkeeper' to target a Thread pool and 'Sleeper' to target each processing threads. Then creates a VPC bundle to process the static maps, those maps could store the indexed reflections of WEB http IP endpoint address. The function and value could be a static or new, it seems more flexibly at this environment. Once finished the task and catched an exception by each Sleeper thread, the Skivvy will do an arrangement of garbage cycling by Hallkeeper. The returning Http response contains many types of feedback processes, for example W3C of buffer, zip, stream etc. For the visit acceleration, VPCS will cache the buffering documents of jpg, html, js, wav etc.

An execution of VPCS application. Boots a VPCS server. Starts a socket accept pool, then waits for the interactions of http hands shaking all the time. Once getting a successful request or rest call from the HTTP web, then forwarding a VPC task to process the business logic. For example, token sessions, data manipulations, cache interfaces and backend operations etc. The executed result could be built as map, then responding in JSON String for feedback. The skivvy will free memory garbages and do the cancellation of each finished socket. Those whose way loops of socket accept is an asynchronized message mode, so It seems VPCS did a well-distribution and waiting for the net requests.

Author YaoguangLuo 稍后优化语法. 具体见第7章 VPCS论文

VPCS调度架构

1 VPCS服务器包含

视觉模块, 处理模块, 控制模块, 资源模块. refer page 396, 394, 392, 383

2 模块标识与查找

每一种模块有各自的名称标识 和 内存标识, 方便精确查找. refer page 492, 493,

3 VPCS服务器架构应用

包含执行者-生产者-造梦者-sleeper, 管理者-分配者-登记者-HallKeeper, 运维者-服务员-清洁员-skivvy 3个模式. refer page 392, 394,

4 VPCS 逻辑价值体现

支持控制与执行分离, 线程与资源分离. refer page 385~389, 486, 490, 492

作者最早设计 VPCS 服务器的动机, 是为了弥补VPC的计算过程观测困难的问题. 因为作者设计的VPC是采用 springboot + mybatis的结构, 底层全是是开源插件的封装, 很多核心源码又不能调试仅仅通过几个log和 try catch给作者带来了无形的压力(作者的思维很简单, 就是自己写个服务器, 能够调试断点从头断到尾), 于是有计划从无到有进行设计一个TCP/IP的 SOCKET 协议做服务器HTTP请求. 作者当时没有想到, 一个这样的小动机给带来了丰厚的回报, VPCS 目前成为了 DNA 元基映射编码算子 的核心组成部分.

作者的rest技术 最早在加州路德大学的实验室 关于德克萨斯大学的香港教授(彭教授的同学) 的网络课, 用camel 进行rpc 部署便接触了. 在加州路德大学的图书馆接触了c语言的http cgi 教材, 之后在很多公司接触了rest的企业级开发如开源的go martini, 开源的 java的 spring mvc, 开源的 jboss的eap, 开源的 springboot 的 application等. 文中的缓存思维, 来自作者做商品搜索时候, 亚米的王浩雷有建议作者使用三级缓存技术来本地加速, 于是作者google搜索下 用了 java jdk的开源 cache manager API. rest的具体paper为 Roy Thomas Fielding的2000年作品, 这里标注下. 罗瑶光

In an early time, the motivation of created a VPCS server, the author did a problem of his VPC open-source project, based on Angular, Mybatis and Springboot, because he couldn't make an inner break-points to do sequence-debugging at that platform. Because all of the Angular, Mybatis and Springboot, were open sources APIs, which were boned and encrypted by DLLs. The author feed so hard to make an efficiency test by only a few fatal logs, tries and catches. Took this problem, he did a plain to build a new TCP/IP web server by using his VPC schedule-method. His purpose was too simple, means only a fully try catchment of his all of the source code. Seem crazy but successful, now a VPCS became an important part of DNA hexadecimal encodings, AOPM VECS IDUQ TXHF.

Before this VPCS creatives, the author did a well job of Alfresco and Its surf distributions, and 32feet at 2009. And a powerlink and modbus protocol at 2010. And a Liferay-theme and md5 CRC circuit, read HTTP cgi by using C at 2012, Camel RPC deployment at 2013. Do note on Golang martini RPC and Angular 1.2 for Kiyor.Cai at 2015, ChinaCache. Java Jboss EAP, JDK third party caches at 2016, Yamibuy, PHP laravel MVC at 2017, 117book. Angular 1.2, spring boot and hibernate at 2017, Intel. And referred Roy Thomas Fielding of his Restful.

PLSQL语言

Gitee 20190820 感想: 设计plsql的编译机,. 做这个项目发现2012年接触的对象化序列化编译技术, 非常有价值. 2016年我搞了2个月jvm的 native虚拟机当时想弄懂jvm openjdk的native函数编译原理, 我之后用go模拟了下, 我用386asm做

plan9 贝尔的 go函数swap 做了3个月实验, 发现编译好的函数名其实只是一串字节码, 这个字符串仅仅需要一个call 就可以在指令中调用了, 我得到一个灵感, 就是传统的编译过程我认为是: 程序 -> 指令 ->机器码 -> 执行. 在编译原理中, 我总结了下其实是: 程序 -> 序列化功能对象 -> 编译机器 -> 调度-> 机器码 ->执行. 有了这些基础, 我开始尝试将我的 plsql 代码 转换成函数先, 然后这些函数进行编译机调度, 然后再进行功能增删改查执行.

另外我还要感谢印度基督大学 的 操作系统 和微机原理 这两门课, (2008)***去主观情绪***.

Implements of PLSQL compiler, notes on Gitee, 2019-08-20.

The author did well fundament of MCU of 89C51 after 2006 (NAA), an Adv MCU of 8085 and Operating system at 2008~2009 (Chrit University), MCU of 485 and Propller (Tyco, 2009) MCU of 28XX at 2010 (Shanghai Electric), and MCU of Stamps (Javline, Basic and Propeller, Callutheran) at 2012~2013. Seem cool about this domain, the author then did experience in 386ASM and Plan9 by using Go, Jvm natives read images, and call jumps simulation of compile-stacks. He considered the traditional compile map of Program-> Structs-> Binary code-> Execution (PSBE), which could be in fact the map of Program-> Sequenced list of functional classes-> Classes translation-> Schedule plans-> Binary code-> Execution (PSCSBE). After a truly proved of DETA PLSQL, he considered PSCSBE could be used in the Creative Languages. Author YaoguangLuo 稍后优化语法

DETADatabasePLSQL 语法

Outline: this document paper makes a pretty explanation of how does DETA database works by using PLSQL Method. At the same time, I will spend more care about the DETA PLSQL runs in the command line or rest call service, with a lot of real world samples.

DETAPLSQLCommands

setRoot:[path];

baseName:[baseName];

tableName:[tableName]:[operation];

getCulumns:[difinition1]:[difinition2]:[difinition3]:[difinition4]:[difinition5];

culumnName:[culumnName]:[dataType];

changeCulumnName:[newCulumnName]:[oldCulumnName];

culumnValue:[culumnName]:[culumnValue];

condition:[operation]:[difinition1]:[difinition2]:[difinition3];

join:[baseName]:[tableName];

relation[operation]:[difinition1]:[difinition2]:[difinition3] ;

aggregate[operation]:[difinition1]:[difinition2]:[difinition3];

CommandsDefinition

setRoot:[path];

The setRoot:[path]; is mostly used for set the database path.

baseName:[baseName];

The baseName:[baseName]; is mostly used for set the current database name in the PLSQL language compiler system.

tableName:[tableName]:[operation];

The tableName:[tableName]'[operation]; is mostly used for set the current table name in current database with the operations. For example tableName:tableName:select; this command will tell PLSQL system, now begin to do the select function section.

getCulumns:[difinition1]:[difinition2]:[difinition3]:[difinition4]:[difinition5] ;

The getCulumns:[difinition1]:[difinition2]:[difinition3]:[difinition4]:[difinition5]:.. .. ..; mostly be used for select columns.

culumnName:[columnName]:[dataType];

The culumnName:[columnName]:[dataType]; mostly be used for create the table columns.

changeCulumnName:[oldCulumnName]:[newCulumnName];

The changeCulumnName:[newCulumnName]:[oldCulumnName]; mostly be used for change the table columns.

culumnValue:[columnName]:[culumnValue];

The culumnValue:[columnName]:[culumnValue]; mostly be used for update the columns value.

condition:[operation]:[difinition1]:[difinition2]:[difinition3]:.. . ;

The condition:[operation]:[difinition1]:[difinition2]:[difinition3]:.. .; mostly be used for

join:[baseName]:[tableName];

The join:[baseName]:[tableName]; mostly be used for select and update of delete with conditions.

relation[operation]:[difinition1]:[difinition2]:[difinition3]:.. . ;

The relation[operation]:[difinition1]:[difinition2]:[difinition3]:.. .; mostly be used for join section condition

aggregate[operation]:[difinition1]:[difinition2]:[difinition3]:.. . ;

The aggregate[operation]:[difinition1]:[difinition2]:[difinition3]:.. .; mostly be used for limit, sort or addition operations.

CommandSamples

select samples

tableName:test:select;

condition:or:testCulumn1|<|20:testCulumn2|==|fire;

condition:and:testCulumn1|>|100:testCulumn2|==|fire;

select where in samples

setRoot:C:/DetaDB;

baseName:backend;

tableName:usr:select;

condition:or:u_id|in|3, 4, 5;

select join samples

tableName:utest:select;

condition:or:testCulumn1|<|20:testCulumn2|==|fire;

condition:and:testCulumn1|>|100:testCulumn2|==|fire; join:stest;

relation:or:uid|==|sid:ussd|==|sssd; relation:and:utoken|=!|stoken:umap|==|smap;

select join samples

tableName:utest:select;

condition:or:utestCulumn1|<|20:utestCulumn2|==|fire;

condition:and:utestCulumn1|>|100:utestCulumn2|==|fire;

getCulumns:utestCulumn1|as|uid::utestCulumn2|as|ussd:utoken:umap;

join:backend:stest;

condition:and:stestCulumn1|>|100:stestCulumn2|==|fire;

getCulumns:stestCulumn1|as|sid|:stestCulumn2|as|sssd:stoken:smap;

relation:or:uid|==|sid:ussd|==|sssd;

relation:and:utoken|=!|stoken:umap|==|smap;

aggregation:limit:2|~|10;

insert samples tableName:test:

insert; columnValue:date0:19850525;

columnValue:date1:19850526;

columnValue:date2:19850527;

columnValue:date3:19850528;

columnValue:date4:19850529;

update samples

tableName:test:update;

condition:or:testCulumn1|<|20:testCulumn2|==|fire;

condition:and:testCulumn1|>|100:testCulumn2|==|fire;

columnValue:date0:19850525;

columnValue:date1:19850526;

update samples

tableName:test:update;

condition:or:testCulumn1|<|20:testCulumn2|==|fire;

condition:and:testCulumn1|>|100:testCulumn2|==|fire;

join:backend:utest;

condition:and:uCulumn3|<|20;

relation:and:testCulumn1|==|uCulumn1:testCulumn2|!=|uCulumn2;

columnValue:date0:19850525;

columnValue:date1:19850526;

delete samples

tableName:test:delete;

condition:or:testCulumn1|<|20:testCulumn2|==|fire;

condition:and:testCulumn1|>|100:testCulumn2|==|fire;

create samples

tableName:test:create;

culumnName:pk:culumn1:string;

culumnName:uk:culumn1:long;

culumnName:uk:culumn1:obj;

culumnName:nk:culumn1:double;

drop samples

tableName:test:drop;

change samples

tableName:test:change;

changeCulumnName:oldCulumnName:newCulumnName;

RealWorldSamplesByUsingDETAPLSQLDatabase

setRoot:C:/DetaDB;

baseName:backend;

tableName:usr:select;

condition:or:u_id|<=|3:u_id|>|7;

condition:and:u_email|!equal|321:u_name|!equal|123;

getCulumns:u_id|as|detaId:u_email|as|detaEmail;

join:backend:usrToken;

condition:and:u_level|equal|low;

getCulumns:u_id|as|sId:u_level:u_password|as|SSID;

relation:and:detaId|==|sId;

aggregation:limit:0|~|1;

Compare Traditional SQL:

```
SELECT u. u_id as detaId, u. u_email as detaEmail, t. u_id as sId, t. u_level, t. u_password as SSID
FROM usr as U
    INNER JOIN (SELECT t. u_id as sId, t. u_level, t. u_password as SSID
                FROM usrToken as t
                WHERE t. u_level equal "low") AS B on U. detaId == B. sId;
WHERE (u. u_id <=3 || u. u_id>7 ) && (u. u_email !equal '321' && u. u_name !equal 123);
LIMIT 0, 1;
```

Acknowledgement

The DETA PLSQL database system source code link:

https://github. com/yaoguangluo/DETA_DataBase

RealWorldSamplesByUsingDETAPLSQLDatabase

```
setRoot:C:/DetaDB;
baseName:backend;
tableName:usr:select;
condition:or:u_id|<=|3:u_id|>|7;
condition:and:u_email|!equal|321:u_name|!equal|123;
getCulumns:u_id|as|detaId:u_email|as|detaEmail;
join:backend:usrToken;
condition:and:u_level|equal|low;
getCulumns:u_id|as|sId:u_level:u_password|as|SSID;
relation:and:detaId|==|sId;
aggregation:limit:0|~|1;
```

Compare Tranditioanl SQL:

```
SELECT u.u_id as detaId，u.u_email as detaEmail, t.u_id as sId，t.u_level，t.u_password as SSID
FROM usr as U
    INNER JOIN (SELECT t.u_id as sId，t.u_level，t.u_password as SSID
                FROM usrToken as t
                WHERE t.u_level equal "low") AS B on U.detaId == B.sId;
WHERE (u.u_id <=3 || u.u_id>7 ) && (u.u_email !equal '321' && u.u_name !equal 123);
LIMIT 0,1;
```

知乎 @Alkaid 罗瑶光

1 德塔PLSQL语言流序模型

是一种从上到下的脚本执行语言. refer page 377,

2 德塔PLSQL语言命令集

包含常用增删改查命令. refer page 406~409, 471, 1035

3 德塔PLSQL语言对数据的操作方式

支持join和 aggregation 高级操作. refer page 419, 431, 435, 438, 447

4 德塔PLSQL语言行批处理模式

可批处理, 可拆分. refer page 1035~1041 将例子写入main, class编译, 然后 bash boot class 即可. 还可以bash 定时批处理.

PLSQL编译机

1 德塔PLSQL编译机价值

用于理解和执行 德塔PLSQL语言. refer page 413, 414

2 德塔PLSQL编译机算子组成

包含常见脚本命令计算算子如 条件算子, 比较算子, 包含算子, 离散算子. refer page 419

3 德塔PLSQL编译机 缓存方式

采用map进行的内部中间数据缓存. refer page 431, 432~

PLORM语言

1 德塔PLORM语言计算逻辑

用于 德塔PLSQL语言进行函数封装. refer page 1003~

2 德塔PLORM语言语法结构

有先后顺序, 需要遵循 德塔PLSQL语言语法. refer page 1019~

3 德塔PLORM语言适用范围 --稍后

对比 德塔PLSQL语言 用于一些不需要配置的nosql的场景, 类似 hibernate 对比 ibatis. refer page 1019~

4 德塔PLORM语言层级

VS hibernate 对比 ibatis的不同, 德塔PLORM语言 另外也是 德塔PLSQL的上层语言. refer page 1019~

德塔的PLORM 和 PLSQL 的引擎出现, 作者开始有信心将其优化成 节点执行的命令行脚本模式, 于是之后的Tin-shell 和 PLTin-shell, PLETL Shell 诞生了. 这个PLETL体系弥补了 当前世界按语言理解方式来模拟神经组织计算的映射空白.

```
Map<String, Object> map= null;
try {
    String plsql= "setRoot:C:/DetaDB1;" +
            "baseName:ZYY;" +
            "tableName:"+ tabKey +":select;" +
            "condition:or:ID|<=|3000;";
    map= ME.SM.OP.SM.AOP.MEC.SIQ.E.E_PLSQL_E.E_PLSQL(plsql, true);
}catch(Exception e1) {
    //准备写回滚
    e1.printStackTrace();
}
return map;
```

德塔 PLSQL

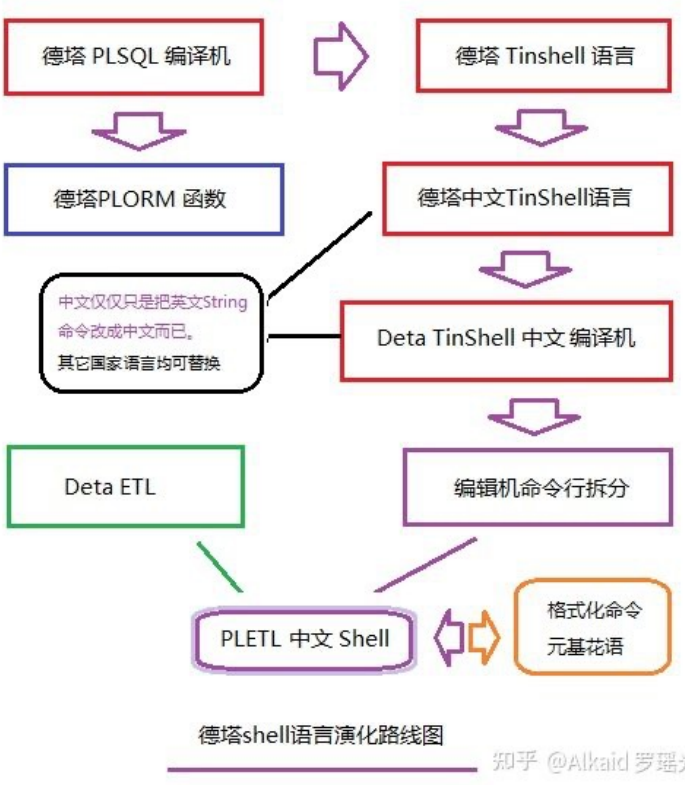
```
select *
from tabKey
where ID<=3000
```

普通带变量 SQL

```
Map<String, Object> map= null;
try {
    PLORM_C orm= new PLORM_E();
    map= orm.startAtRootDir(rootPath).withBaseName(baseName)
            .withTableSelect(tabKey).withCondition("or")
            .let(conditionSubject).lessThanAndEqualTo(conditionObject)
            .checkAndFixPlsqlGrammarErrors()
            .checkAndFixSystemEnvironmentErrors()
            .finalE(unTest).returnAsMap();
    //map= org.plsql.db.plsql.imp.E_PLSQLImp.E_PLORM(orm, true);
}catch(Exception e1) {
    //准备写回滚
    e1.printStackTrace();
}
return map;
```

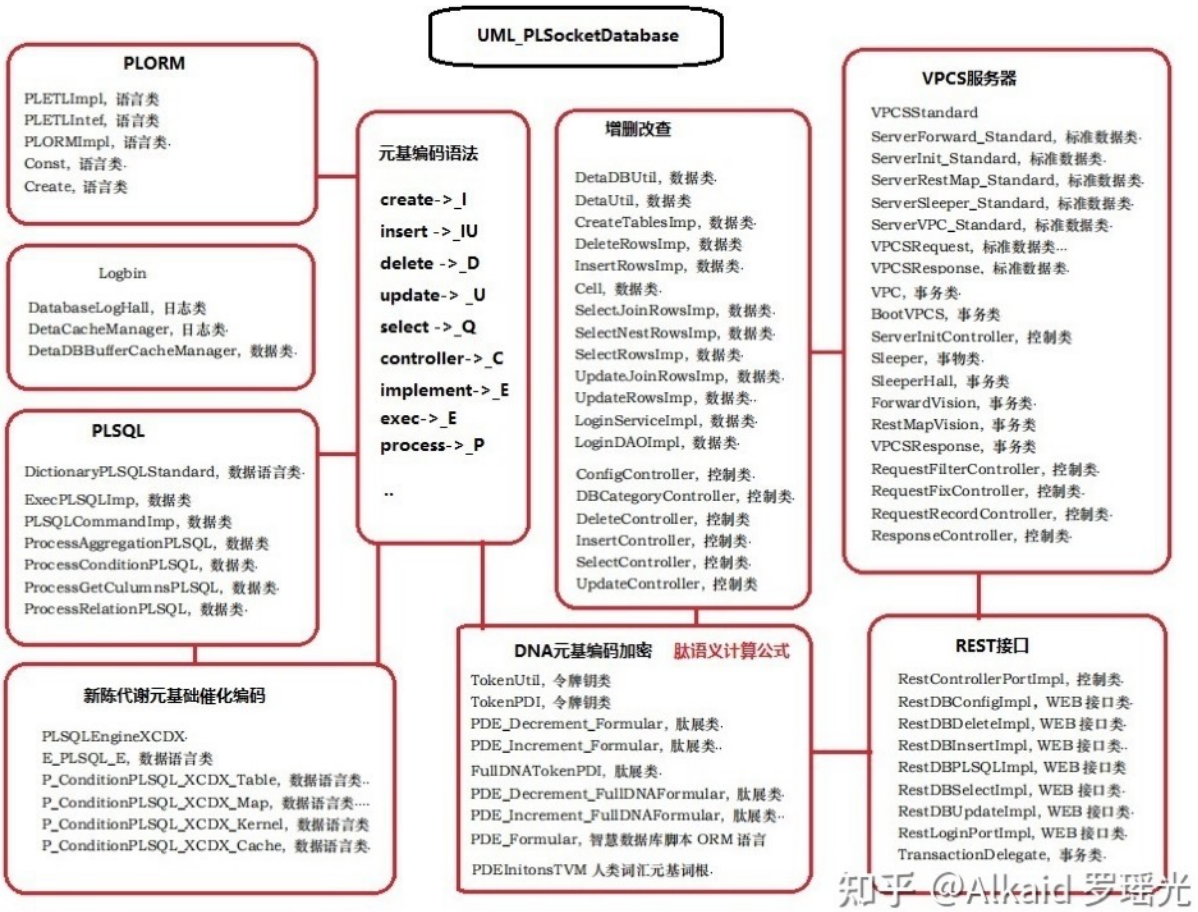
德塔 PLORM

知乎 @Alkaid 罗瑶光



德塔shell语言演化路线图

知乎 @Alkaid 罗瑶光



灾后重建

1 德塔数据库包含logbin 系统.

refer page 398,

2 支持logbin 加密写操作

德塔数据库包含logbin 系统基于单个写操作进行log保存 并行加密成文件. refer page 399

3 采用logbin 时间戳组合 key 标识

单个写操作用时间戳作和写增量序列进行对应标识, 避免混乱. refer page 399

4 支持rollback 逻辑

德塔数据库包含logbin 系统 并支持热备和错误写 实时rollback 检测. refer page 398

德塔的logbin系统, 一开始是设计在try catch 中, 因为德塔数据库融合了cache 和 DMA两种存储系统, 于是, 作者将 logbin 的 rollback进行先内存模拟执行写操作, 成功后再执行物理写操作, 并记录操作日志. 如物理写操作还失败, 就 rollback 到上次写请求. 这种 3步logbin机制, 作者认为 高安全性.

于是开始思考

1 常规数据库因为侧重企业级安全，所以数据IO性能低，体积大，组织多，日志逻辑复杂并严谨，我的数据库因为是嵌入式作业，侧重缓存和数据调度逻辑，如果逆向hsql 或者 mysql 研发方式去尝试替代，动机是化简完全可控的企业级应用方案，BPM思路如何开始。这个定义为研2以上难度。

2 hvpcs 服务器目前我使用的是java，因为socket的写操作中断是java层计算，一旦exception，捕获困难，我目前的解决方式是 skiivy 的调度 和异常线程clean掉，这种方式虽ok但耗性能，如何在java 层写 jvm 的 线程控制逻辑，目前的解决方式有JNI的 C整体替换掉java，是否有更高效简介的逻辑处理这个问题。这个定义为大4以上难度。

3

4

5

6

第五章_德塔数据结构变量快速转换

基础应用: 元基催化与肽计算 编译机的内存分析机

知识来源,

数据结构《Data Structure Using C》仅此而已. 作者敢撰写这本书籍,至少70%能力来自Rohini.V.

内存的结构

1 德塔数据结构变换--稍后

最早归纳来自对 雪球新浪的股票数据 web页抓取进行的String格式统一. refer page 508, 528

2 基于String的格式统一

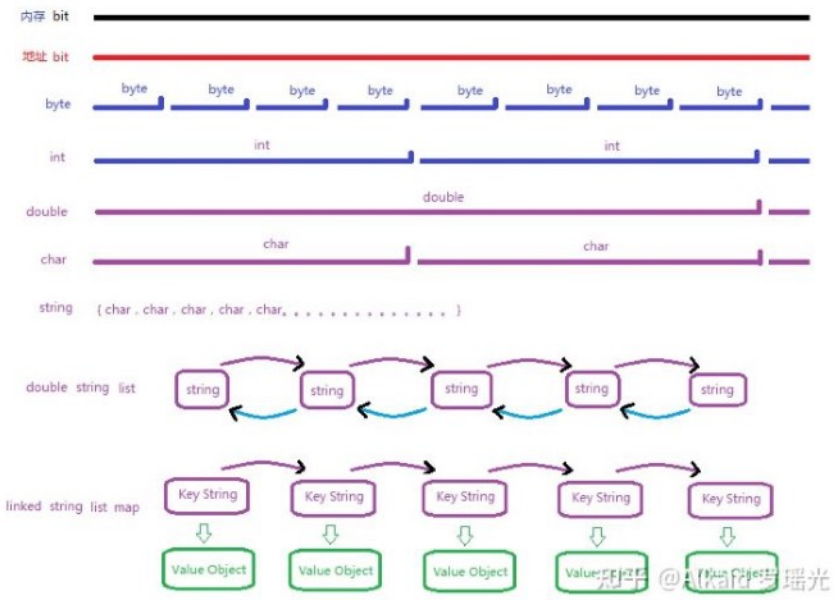
, 然后逐步进行文本数据在计算过程中的状态进行分类扩展归纳. refer page 532, 535 string的展开应用归纳过程: (走四方的xml变换 和 亚米的list<string[]> 还没有迫使作者产生研发swap的动机.)早期作者在设计股市抓取, 发现 雪球, 新浪, 东方, 财富的网页股市数据, 字符串的编码不统一. 于是准备写个自适应的编码变换解析, 根据rest get return的字节码标识来自动变换. 作者的股市string数据在计算过程中要进行加速计算, 于是开始将string[] 变换成list<string>, 这样才有 iterator<string>的buffer加速计算模式, 作者后来在写DETA parser的时候, 将string进行 string builder 来做buffer加速计算, 于是 基于 string[], list<string>, iterator<string>, string builder的最早4个data swap 快速变换包引擎开始设计了. 于是就把所有的数据结构变量都扩展归纳了.

3 高频内存结构的快速互换

于是产生array, StringBuilder, iterator, map, 4种 高频内存结构的快速互换. refer page 499, 536, 515, 520

4 统一归纳和快速变换

最后进行对所有常见数据结构进行统一归纳和快速变换. 作者的研发基础来自2008年 在印度基督大学的C语言数据结构实验室课程. 讲课教授 Rohini. V refer page 492~常见数据结构类型, 罗瑶光画图如下



数据的结构

1 德塔数据结构梳理

完整依据 C语言数据结构 思维进行归纳refer page 无

2 数据变换模式归纳

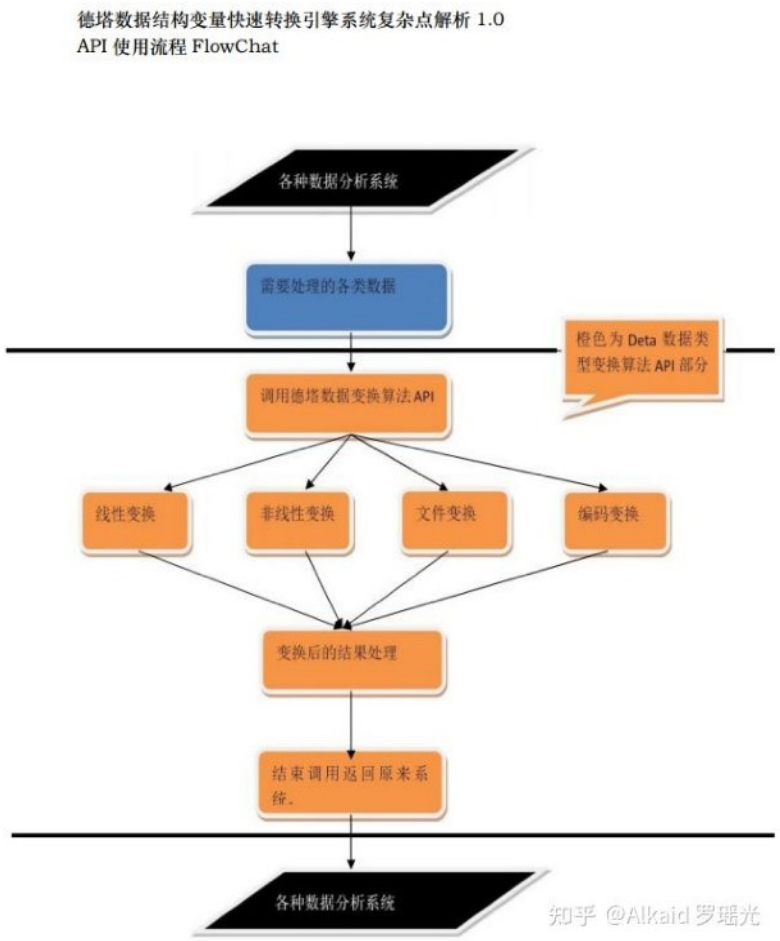
包含 array, String, struct object, hash, map list, tree, buffer的数据变换模式. refer page 499, 535, 527, 507, 520, 516, 537,

3 变换主体

德塔数据结构不包含数据的计算逻辑变换, 仅仅包含数据类型的载体变换. refer page 498

4 计算表达形式

数据类型的载体变换通过接口形式表达. 广泛用于工程中. refer page 498



罗瑶光画图

类的结构

1 接口模式

德塔数据结构的类, 采用VPCS的静态接口模式设计. refer page 492~

2 封装模式

每一种相同数据类函数封装在同类的文件中. refer page 492~

3 类的包含属性

每一个类 主要包含数据变换文件, 数据变换的纠正文件, 数据变换的索引文件. refer page 492~

计算优化-转换加速

1 计算优化方式

数据变换的索引文件, 通过元基花索引24组染色体注册, 进行语言调用加速. refer page 下册597 StaticFunctionMapU_VECS_E

2 变换加速方法

数据变换采用静态函数, 加速了function call. refer page 492~全章

3 分类与模式

数据变换的函数 根据功能进行了分类, 于是静态函数文件形成了balanced静态函数集树模式. refer page 下册274 第十六章

不规则对象的变换

1 属性包含

不规则对象的变换主要包含 邻接矩阵array变换和 类复制. refer page 521

2 跨格式变换

邻接矩阵array变换 如 跨格式变换, 如（xml已剔除）, json, officerefer page 558, 516, 503

3 对象类复制

如 deta的TinMap class和 Objectrefer page 527, 881

4 申明

xml和json, 德塔不做加工, 仅仅用google的Gson包引用. refer page 516

计算需求-场景变换

1 主要应用

德塔数据结构的场景主要应用在网页html数据抓取, 文本数据计算refer page 508, 492~

2 价值体现

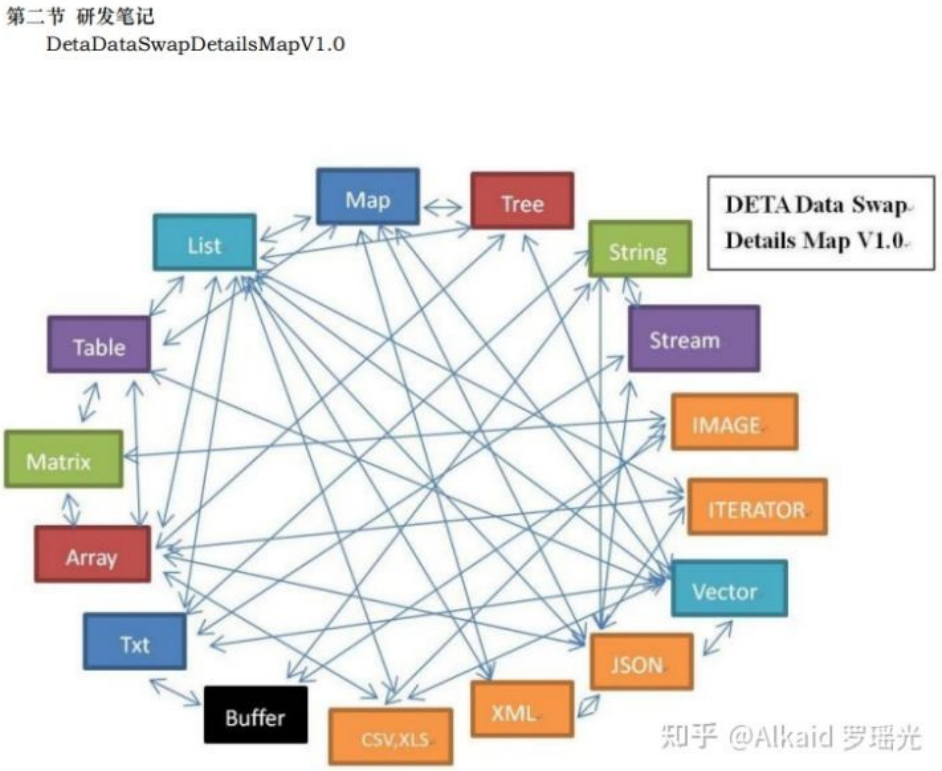
html数据 主要体现在文字的编码格式变换, 加密变换, 和http response的内容载体变换如json. refer page 508, 555,

3 文本数据类应用

计算主要体现在 map和array的变换, 与 list 和 array的变换, 用于字符串排序加速. refer page, 499, 516, 520

4 可加速的计算变换方法

在德塔分词场景中体现在另外String与 StringBuilder的加速变换. refer page 536



罗瑶光画图

20220409 截至目前作者接触过得所有计算机变量载体和object对象类型总数, 远远的超过图中的列举类型, 作者发现这些类型, 都或多或少有图中的类型的影子, 或者包含有图中类型的类似结构. 比如枚举作者没有画图, 枚举便类似 struct, 同时又具有array的地址连续性, 等作者不多做解释了, 计算机语言书籍 有枚举的标准定义.

每次想到数据结构我能有这种造诣,溯源下,还是对数字逻辑的极其深刻的理解,于是又想起了JPSir.

JPSir was my teacher, every time I thought of him. My eyes such as the lake.

The first story

I still remembered a class where JPSir let us do the assembly language for delete. He said we can't use 8086/ or other higher version to do this logic. So, we try to use 8085. After I searched all the ASM commands, I did not find any resource about delete. Then JPSir came and asked, how does computer make a delete? I got results, using n's complement and implement and added 1, if got carry, then did a complement and added 1 again. Then he got a smell, 8086's delete was truly promoted with this method. After this session, I strongly knowing that science was an evolutional inheritance. For everything we know how to use it, and we also need to know how to made it.

The second story

This was a computer IC class and we needed to do a runtime design of CPU, to make a calculation of each ASM command processing, we also need to draw the state graph. When I finished the question, I saw him toward with me and looked carefully with my graph, after a while and asked me where did the clock comments? I said at the left part, he angered with

me and said: why you write at the left part? all the book, all the resource was for the right-side. Suddenly I knew my mistake, and speedily changed to the right side. After this session, I strongly knowing that science was a rigorous and an accuracy, everything we know how to use it, but we could not to make change of it.

字符串编码变换

1 输入法编码

--当前的文档编辑输入法有一些误区, 当进行频繁格式转换的时候出现的乱码会无法还原, 当这个无法还原码在全文哪怕就出现一次, 那么这个全文转换会全部出现乱码问题导致无法保存和再转换, 最常见的是 UTF GBK UNIcode 转换产生的无法还原码. 所以任何时候在工程中, 首先要做的便是统一字符集. 于是用 DETA swap 进行统一预处理.

2 文件格式

的问题 -- 不同的文件格式, 存储数据中的片段资源内容都是不一样的, 特别是一些需要质量控制的矢量图存在的文档, 特别要注意在保存前的逻辑计算模型, 避免图片失真, 文字模糊, 体积庞大等各类问题. 于是用DETA swap 进行tookit 格式参数调优预处理.

3 汉字 和 古拉丁文词汇进行元基编码变换

-- 稍后--因涉及罗瑶光核心技术-- 公开下2022-2023年的部分技术，避免自己垄断

1 拉丁字符表

ABC-DEF-GHI-JKL-MNO-PQR-STU-VWX-YZ

2 元基字符表

AOPM-VECS-IDUQ-TXHF

3对应表

ABC-DEF-GHI-JKL-MNO-PQR-STU-VWX-YZ

VSE-DID-IXX-OMC-IDY-IUM-ASS-VSA-IH

SID-CVO-HQS-IVI-MTQ-QDS-EQU-UDX-UM

罗瑶光先生通过大量的罗曼早期和拉丁语意字符进行溯源统计，根据严谨的闪米特工匠类和农佃类象形文字符

定了2组双元根拉丁字符元基解码表。该表含有罗瑶光先生主观对事物的认知方式。工程应用切记

时函数滤波变换

时函数的溯源

关于虚空与无理级函数铺垫

在数学的范畴，来理解关于虚空与无理级函数，可以用 ‘假设存在’ 和‘伪命题’ 两个词组来对应
这样虚空便是假设存在的 对象。
无理级意味着这些虚空对象们的关系的状态 伪命题。

在数学论证中，引入虚空和无理级的概念，可以增加方程论证的方法和条件。最后用一些数学归纳法可以得到浓缩的理解力。

虚空在 时函数中的作用 偏向于工程数学领域 我用i来代表，因为复变积分有假设 i * i = -1的例子 我们可以假设 i.x(-1) * i.y(-1) * i.z(-1) = 1 不要套近乎。
1 增加计算的维度
2 减少偏移的过程
3 分解复杂的数据

无理级在时函数中的作用 偏向于离散数学领域 我用l来代表，目前还没有，但我的dna元基催化与肽计算 著作权版本 之前已经有了无理级关联的明确文字描述。
比如 l(f(x))= l.f.x , l(x + y) = f.x+f.y,
1 减少复杂的计算指令
2 得到关系的最简模型

或许还有其他价值，因为脱离了我的76页时函数推导，这里略。
上面组合举例 虚空无理 伪命题方程 来求解 l 的 值，偏向于代数学
i.x(-1) * i.y(-1) * i.z(-1) = 1 = l.x.i + l.y.i + l.z.i

时函数处理虚函数和纯函数的扩展思绪

真数虚空变换-我举个例子先，告诉大家用法。
true = 1
d.true = d.f(cosA * cosA + sinA * sinA)
= d.f(cos(A+B) * cos(A+B) + sin(A+B) * sin(A+B))
= l.d.f(cos(A+B)) + l.d.f(cos(A+B)) + l.d.f(sin(A+B)) + l.d.f(sin(A+B))
= l.d.f(cos(A+B)) + l.d.f(sin(A+B))
= l.d.f(cosα*cosb - sinα*sinb) + l.d.f(sinα*cosb + cosα*sinb)
= l.f(cosα'* cosb + cosα * cosb' + (-sinα)'* sinb + (-sinα) * sinb')
+ l.f(sinα'* cosb + cosα * sinb' + cosα'* sinb + cosα * sinb')

= l.f((-sinα)* cosb + cosα * (-sinb) + (-cosα)* sinb + (-sinα) * cosb)
+ l.f(cosα* cosb + cosα * cosb + (-sinα)* sinb + cosα * cosb)

= l.f(-2sinα * cosb -2cosα * sinb) + l.f(2cosα * cosb -2 sinα * sinb)
= l.l.f(-2sin(α + b) + 2cos(α + b))
= l.l.detα.cos(α + b) - l.l.detα.sin(α + b) + N
= l.l.detα.cos(C) - l.l.detα.sin(C) + N

时函数的优化FFT滤波算法

罗瑶光先生2024公开了 {+- 1 +- 2*CosA*CosB} 的噪音能公式。和它的真实应用方式。

1 核心逻辑是2019的量子公式
|tero(x)>+|tcol(x)> = detα (t1-t0)*m(t) / detα1(t1-t0)*p(t)

2 矢量分解成
ft + fd * fr = E
fi = ft * fr / fd
ft = fi * fd / fr

3 处理带能FFT蝶形内核。

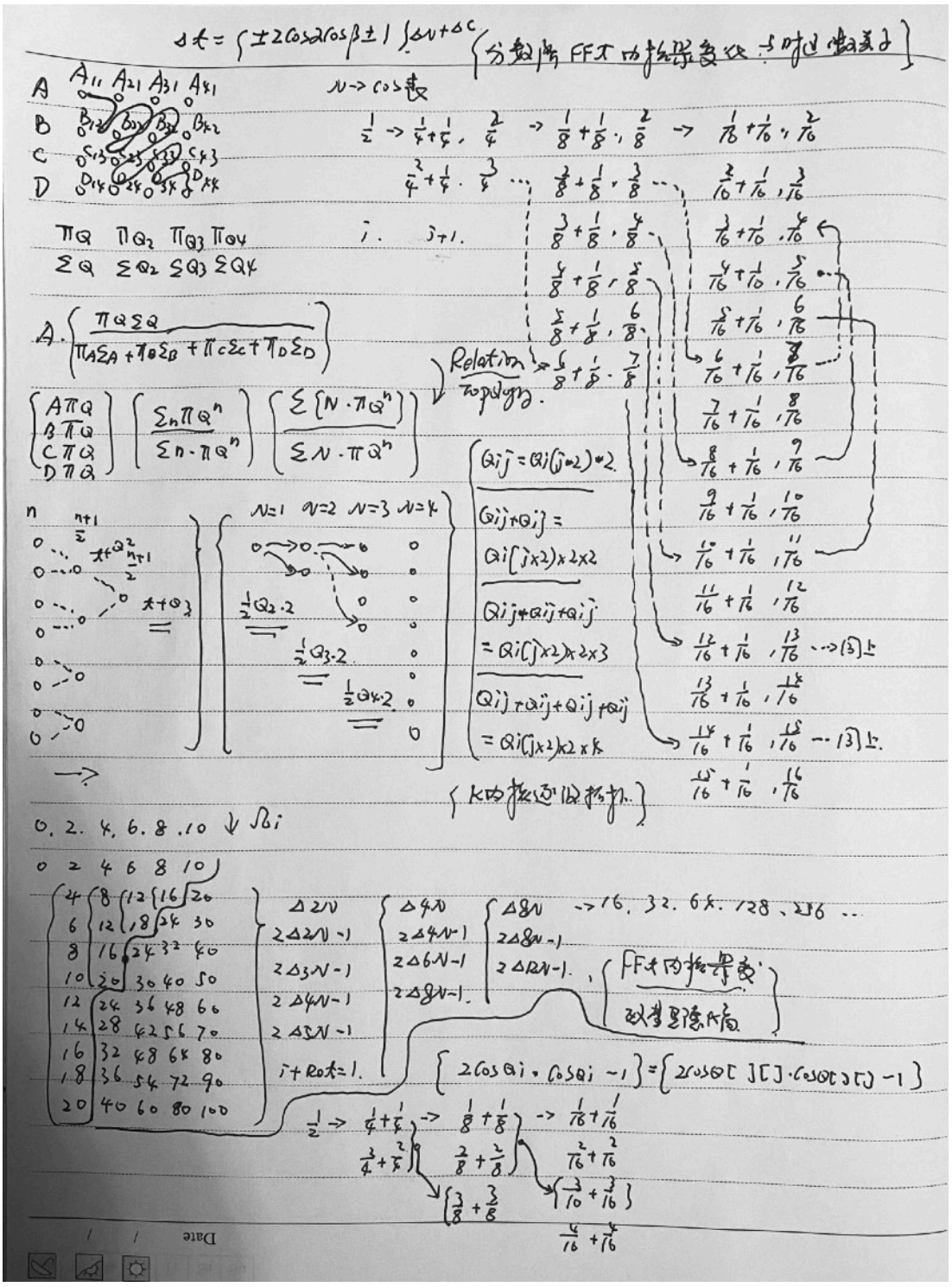
E = [f(d)] * [f(d)] * i * di

beta m = 1/ (2 * sqrt(i)) * sin(A - B)

4 最终得到

{+- 1 +- 2*CosA*CosB}

时函数微分进行log熵化2倍角卷积核 逻辑图



上图中在进行傅立叶卷积的计算时，每一个n的i区间的计算量的算子是可以拓扑到 i+1的层面，因为互联网和教材提供的是蝶形和差化积公式拓扑的，导致矩阵ij算子不能拆解，现在用2倍角公式替代便能优化一些关于fft层面的实际问题。关于dct优化这里忽略先。

图中左上再考虑斜方卷积变换的时候，需要思考每一个子集的内变换，图上右关于1/2 子集 与 一个 分数阶的

i+1的 1/4+1/4 再到 i+2 的 1/8+1/8+。。。 这里的拓扑方式呈现了2倍角的变化规律，于是作者将 原fft 进行 和差化积 进行替代实验开始。

--图片优化傅立叶和差化积蝶形卷积核，生成On计算模型算法如下

```
package S_A.linePage;
//罗瑶光2026年1月15日基于世面上的库里图基FFT递归算法进行 熵化蝶形 计算序列重排 去 递归 变换 重新优化。
public class LDFFT{
        float [][]cosNT;
        float [][]sinNT;
        float [][]detaT;
        float [][]detaTs;
        float [][]detaT2;
        float [][]detaTs2;
    //初始一个计算核，用于 fft蝶形生成dct余熵2倍角聚变核 做 时流积 来计算频率。速度-O(n)
    public void initLog2ktfft(int inputLength) {
    //float[] fxets = new float[input.length];
    int size = inputLength;
    int log = 2;
    float pi = (float) 3.1415926;
    //这个for for循环init可以拿出去new， 提高实时计算效率
        cosNT = new float[inputLength + 1][inputLength * 2 + 1];
        sinNT = new float[inputLength + 1][inputLength * 2 + 1];
        detaT = new float[inputLength + 1][inputLength * 2 + 1];
        detaTs = new float[inputLength + 1][inputLength * 2 + 1];
        detaT2 = new float[inputLength + 1][inputLength * 2 + 1];
        detaTs2 = new float[inputLength + 1][inputLength * 2 + 1];
                for(int i = 0; i < inputLength; i++) {
                        for(int j = 2; j < inputLength * 2; j*=2) {
                                cosNT[i][j] = (float) Math.cos(i * pi / j);
                                sinNT[i][j] = (float) Math.cos(i * pi / j);
                        }
                }
                for(int i = 0; i < inputLength; i++) {
                        for(int j = 2; j < inputLength * 2; j*=2) {
                                if(i <= inputLength/4 && j <= inputLength/2) {
                                        detaT[i][j] = cosNT[i << 2][j << 1];
                                        detaTs[i][j] = sinNT[i << 2][j << 1];
                                        detaT2[i][j] = cosNT[i << 1][j << 0];
                                        detaTs2[i][j] = sinNT[i << 1][j << 0];
                                }
                        }
                }
        }
        float[] dlt;
        float[] dlts;
```

```
    float detaET[][];
    float detaETs[][];
    //初始一个计算核，用于 fft蝶形生成dct余熵2倍角聚变核 做 时流积 来计算频率。速度-O(n)
    public void initLine2ktfct(int inputLength) {
    int size = inputLength;
    int log = 2;
    float pi = (float) 3.1415926;
    //这个for for循环init可以拿出去new， 提高实时计算效率
    cosNT = new float[inputLength][inputLength * 2];
    sinNT = new float[inputLength][inputLength * 2];
            for(int i = 0; i < inputLength; i++) {
                    for(int j = 2; j < inputLength * 2; j*=2) {
                            cosNT[i][j] = (float) Math.cos((i - 1) * pi / j);
                            sinNT[i][j] = (float) Math.sin((i - 1) * pi / j);
                    }
            }
       detaT = new float[size][size];
       detaTs = new float[size][size];
       detaET = new float[size][size];
       detaETs = new float[size][size];
   //log - section - loop
   while(log < size) {
   //loop
   double Ei = 0;
   int logd = log >> 1;
   int logi = log << 1;
   //double detaT = cosNT[0][log * 2]; // 1/16
   for(int j = 0; j < logd; j++ ) {
   //去掉时函数的2倍角代换进行内核化T熵查询即可，
   //double detaT = cosNT[j << 2][log << 1];
   detaT[j][log]= cosNT[j << 2][log << 1];
   detaTs[j][log]= sinNT[j << 2][log << 1];
   detaET[j][log]= 1 + detaT[j][log];
   detaETs[j][log]= 1 - detaT[j][log];
   //因为我设计内核是从0开始，而这里幂计算是1开始对2的倍数熵，
   //所以大家用于各类具体实际工程记得去校准一下，
   //section - loop
   }
    log = logi;
}
log = 2;
dlt = new float[inputLength];
dlts = new float[inputLength];
for(int i = 0; i < inputLength; i++) {
    dlt[i] = 1;
    dlts[i] = 1;
}
while(log < size) {
   //loop
```

```
        double Ei = 0;
        double Eis = 0;
        int logd = log >> 1;
       int logi = log << 1;
       for(int j = 0; j < logd; j++) {
          for(int i = 0; i + log < size; i = i + log) {
                  Ei = dlt[i + j] * detaT[j][log];
                  Eis = dlts[i + j] * detaTs[j][log];
                  dlt[i + j] += Ei;
                  dlt[i + j + logd] -= Eis;
                  //因为减熵是不规则的补码加乘逻辑，于是剔除，
                  //因为剔除了蝶形的倍频减熵，于是用sin来补充迪摩根互补律。
                  dlts[i + j] -= Eis;
                  dlts[i + j + logd] += Ei;
                     }
       }
        log = logi;
    }
        }

        //fft蝶形做dct余熵核2倍角聚变来计算频率。速度-O(nlogn)
        public float[] doLog2ktfct(float[] input) {
                 float[] output= input.clone();
                 int log = 2;
                 int size = input.length;
                 while(log < size) {
                double Ei = 0;
                int logd = log >> 1;
               int logi = log << 1;
               for(int j = 0; j < logd; j++) {
                   //section - loop
                  for(int i = j; i + log < size; i = i + log) {
                          Ei = output[i] * detaT[j][log];
                          output[i] += Ei;
                          output[i + logd] -= Ei;
                             }
               }
                log = logi;
            }
            for(int i = 0; i < output.length; i++) {
                output[i] = Math.abs(output[i]);
            }
            return output;
        }

        //fft蝶形生成dct余熵2倍角聚变核 做 时流积 来计算频率。速度-O(n)
        public float[] doLine2ktfct(float[] input) {
                 float[] output = new float[input.length];
                 for(int i = 0; i < input.length - 1; i++) {
```